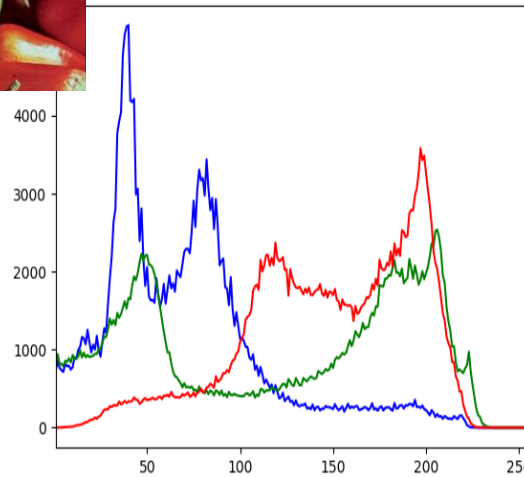


Image Processing - Lecture 6

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk



Reading:

Szeliski - Section 3.1.4 (limited)

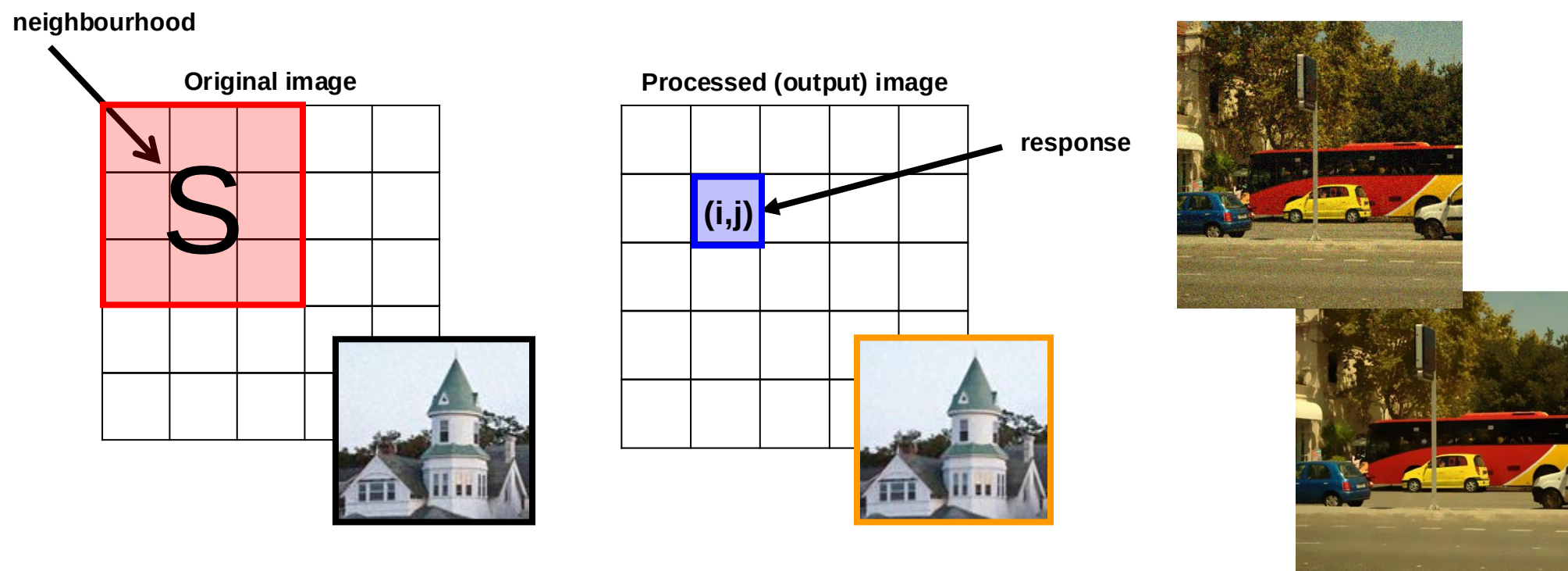
Gonzalez/Woods - Sec 3.3

Solomon/Breckon - Sec 3.4

Histograms and Histogram-based Image Enhancement

Slide material acknowledgements: Fisher; Breckon; Ivrisimtzis; Atapour

Recap ... Spatial Filtering



Previously: image neighbourhood and spatial filtering = local image transforms.

More advanced filters:

- Bilateral Filtering
- Non-Local Means

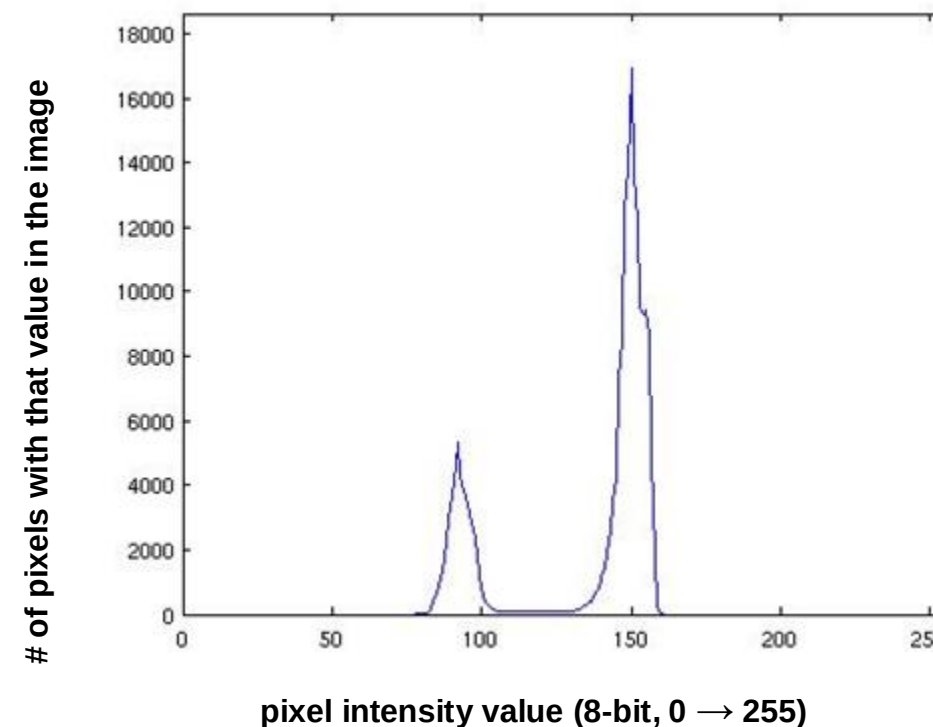
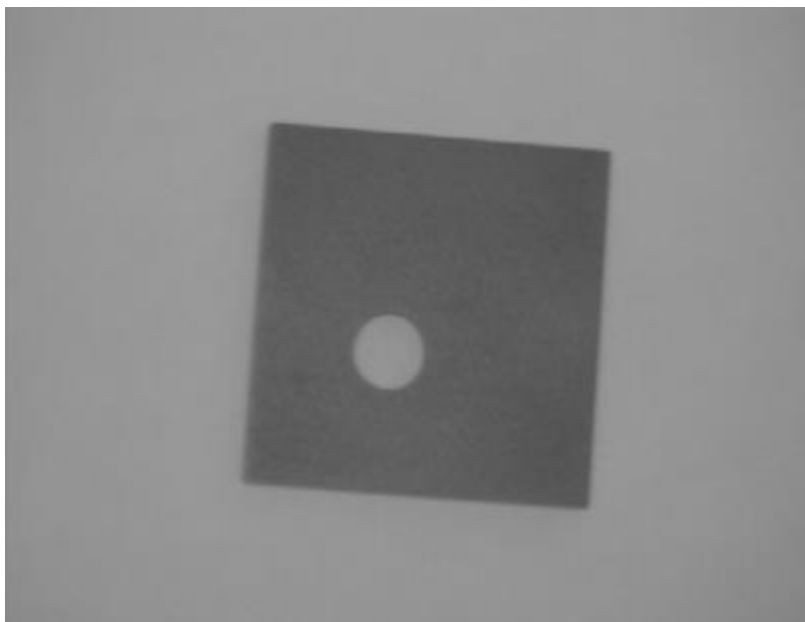
Today: global image transforms based on image histograms

(i.e. the statistical distribution of pixel values)

What is a Histogram?

The **histogram function** (or histogram distribution) of an image is a function defined over all possible intensity levels, e.g., [0-255].

For each intensity level, the **value** is the **number of pixels with that intensity**.



Constructing a Histogram

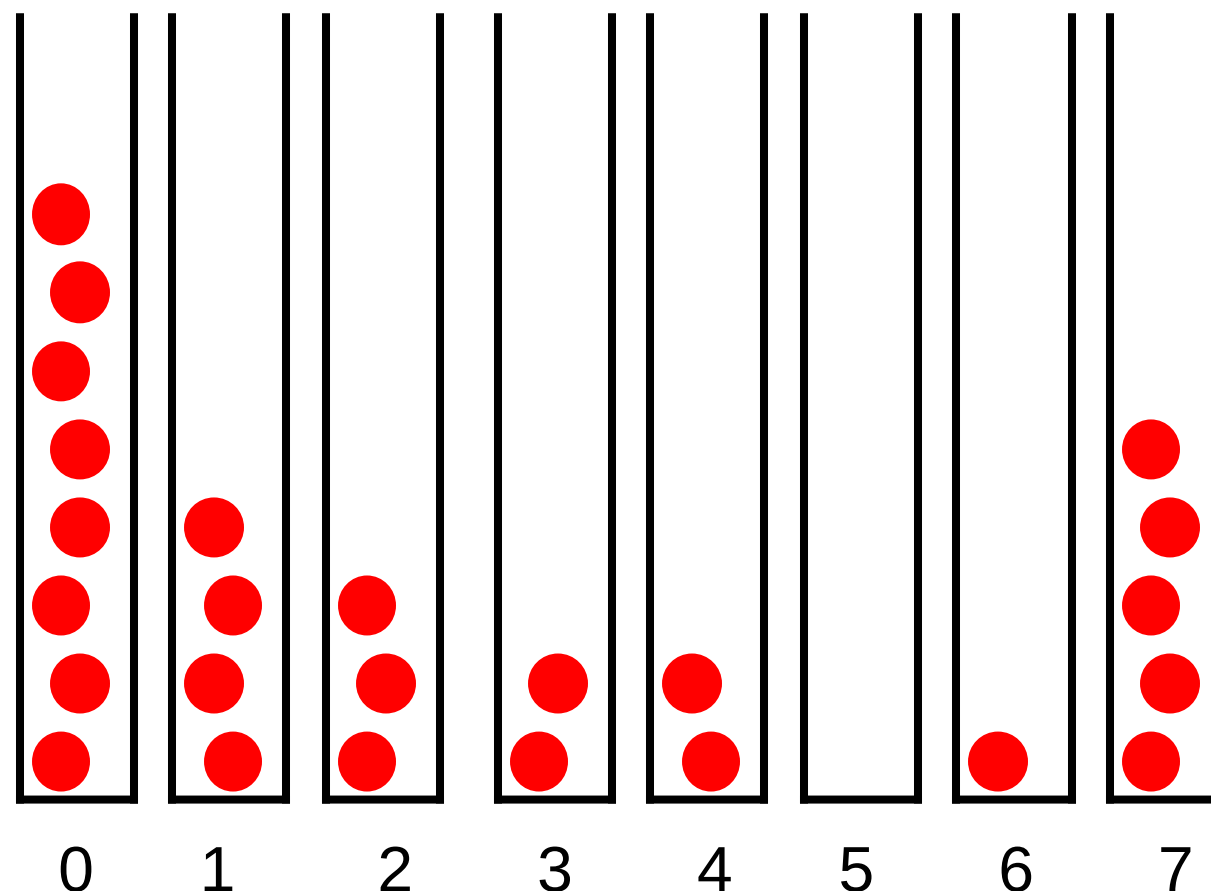
- Simply count occurrences of each grayscale intensity value (or each R,G,B colour value for colour images).
- Algorithm pseudo-code:

```
initialise all histogram array entries to 0  
  
for each pixel I(i,j) within the image I:  
    histogram[I(i,j)] = histogram[I(i,j)] + 1  
End for
```

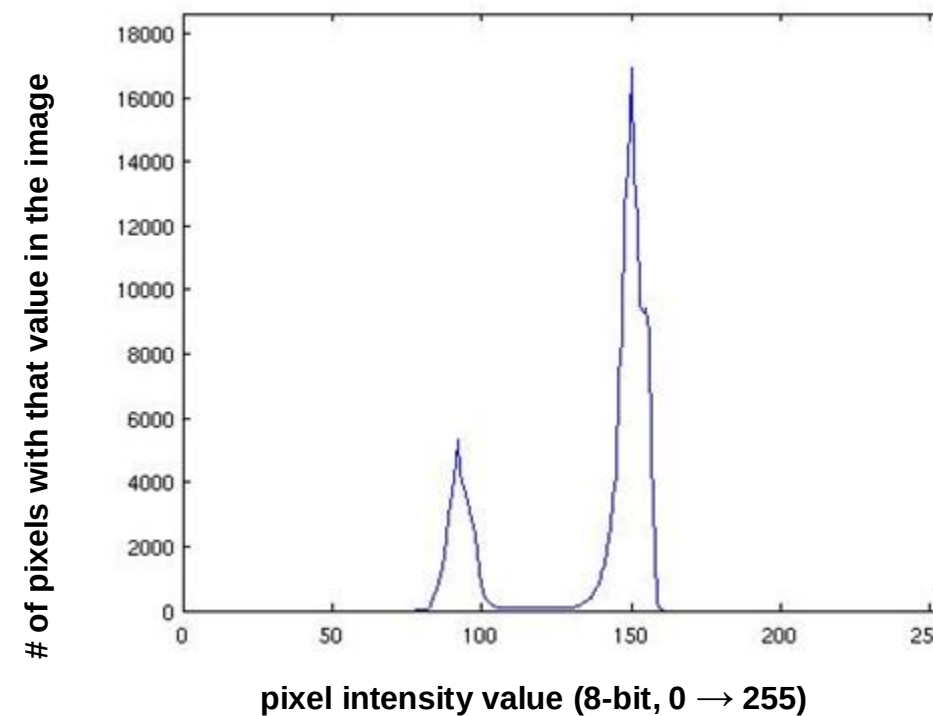
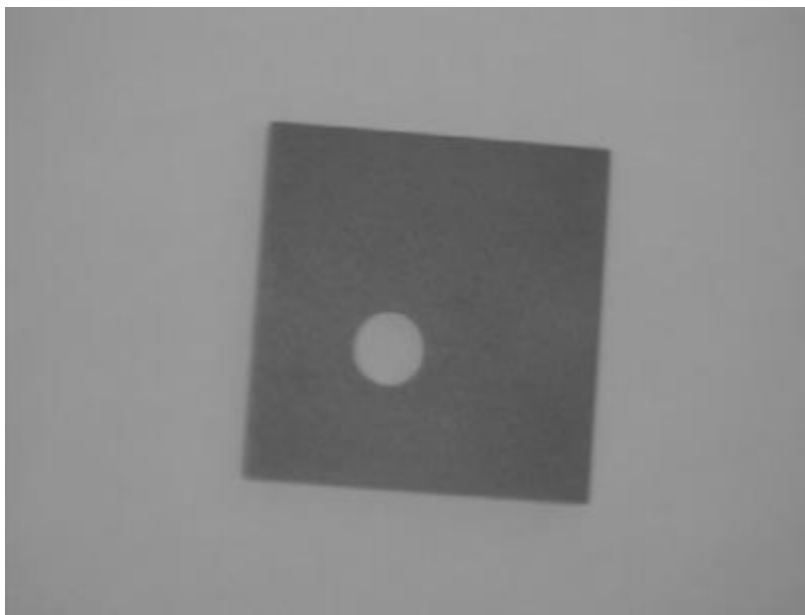
Constructing a Histogram

- Iterate over image, count pixel values into histogram array.

0	7	3	2	3
0	0	0	6	7
7	7	2	2	0
1	1	0	4	1
0	0	7	4	1



Understanding Histograms



❑ What is the max possible value of the histogram x-axis? **255**

❑ What is the max of the histogram y-axis? **17,000**

❑ What part of image is the: -- small peak in the histogram? **Foreground**

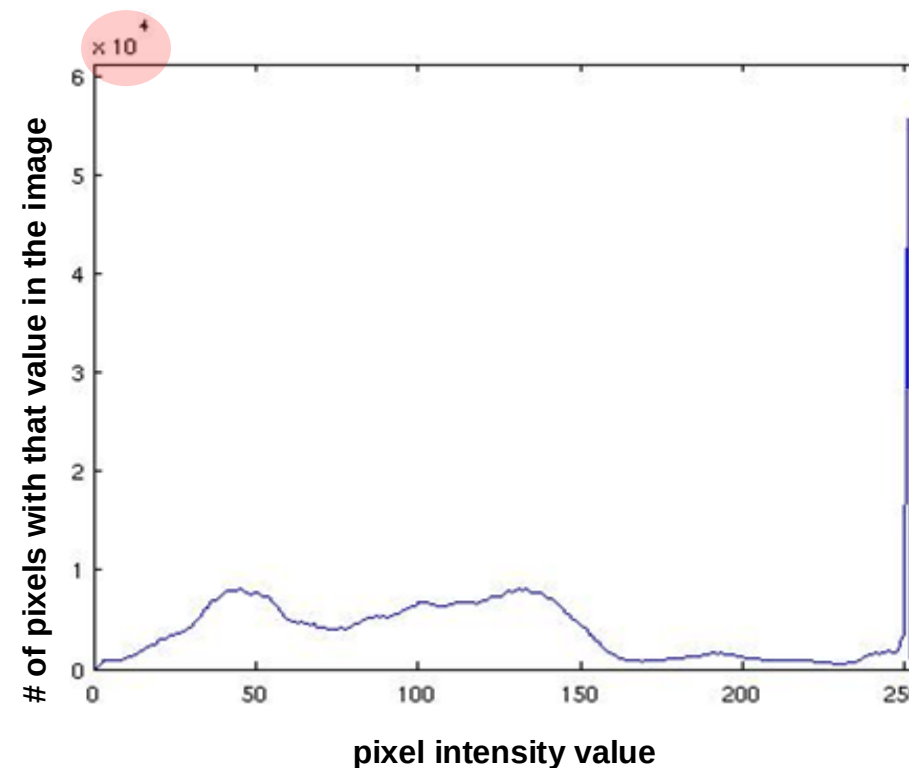
-- large peak in the histogram? **Background**

Histograms



<https://youtu.be/VFkHM5XrHXY>

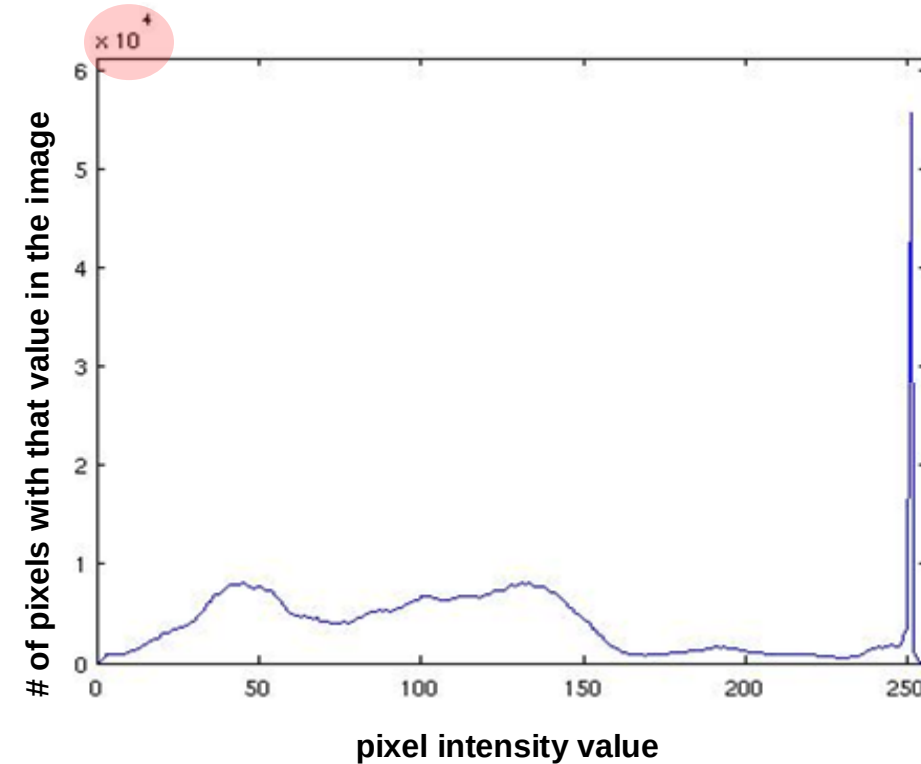
More Complex Scenes ...



More complex pixel distributions in the histogram.

- Here, separation of darker objects (range ~25-150) from the light portions of the scene (>250).

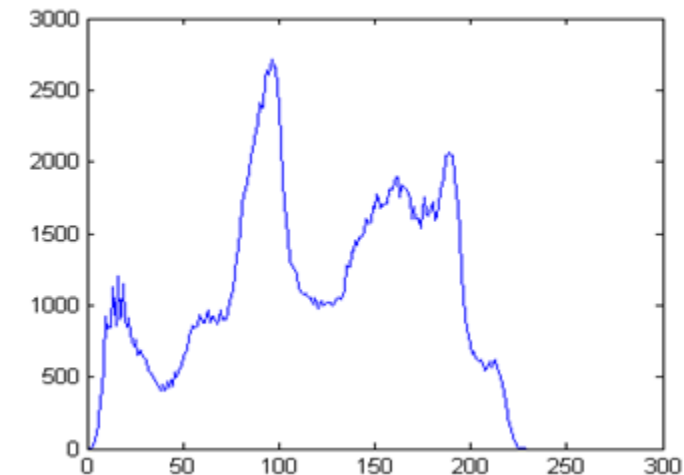
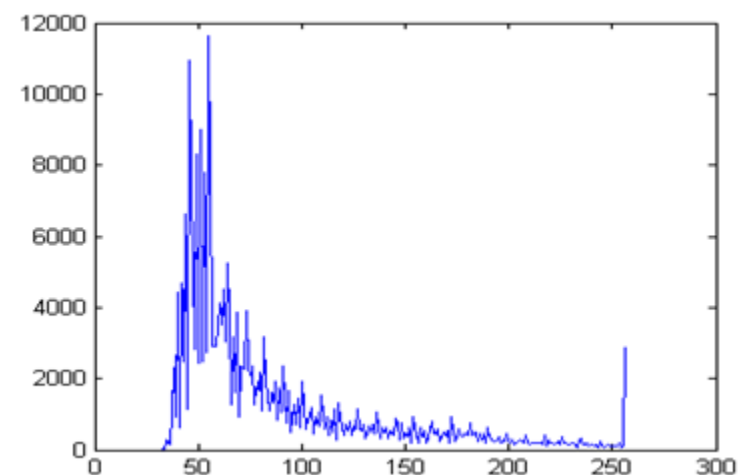
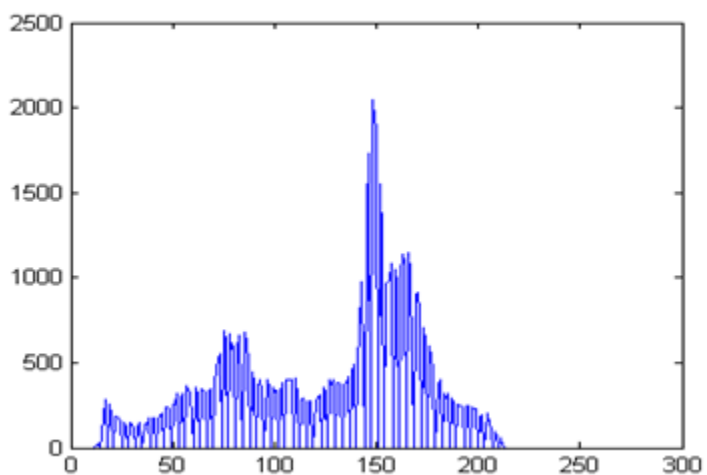
More Complex Scenes ...



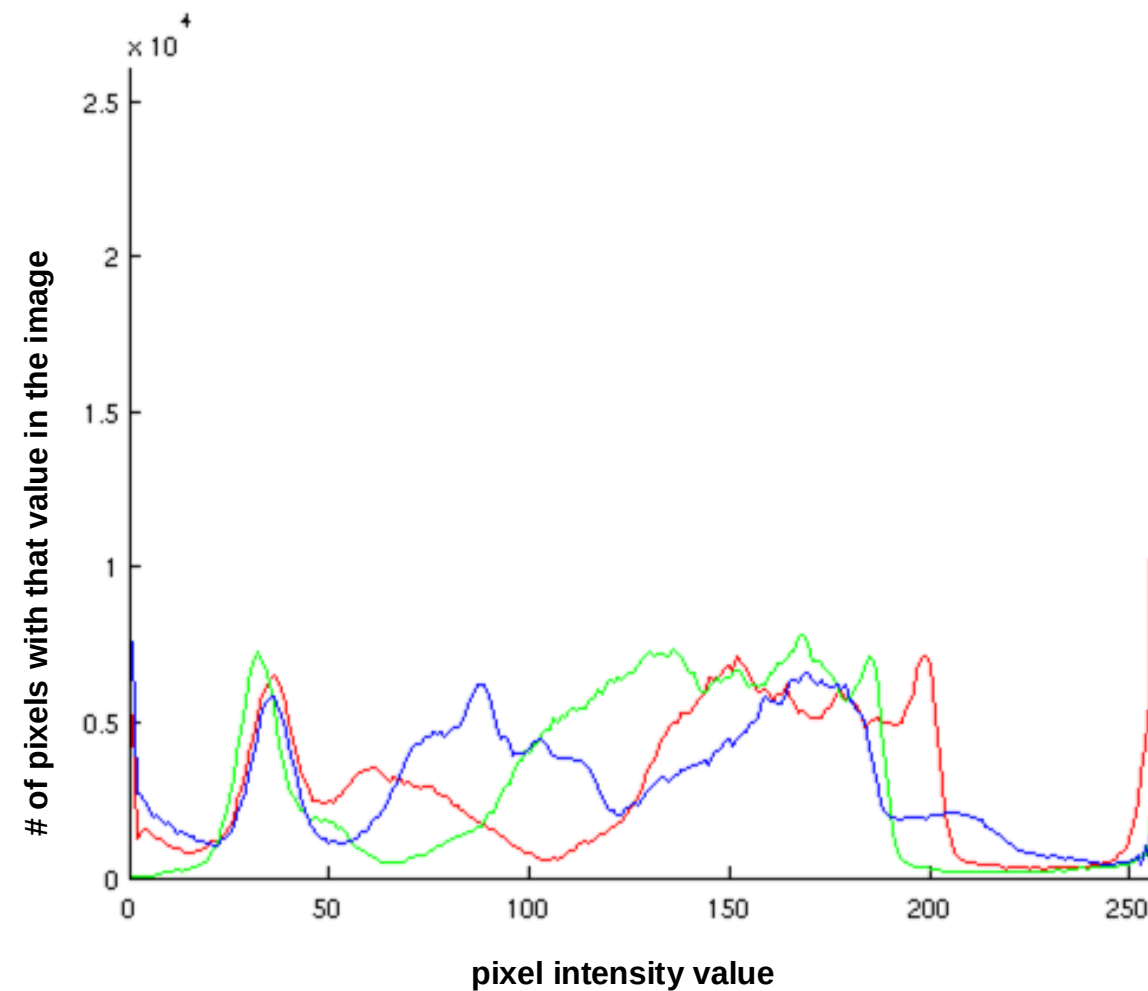
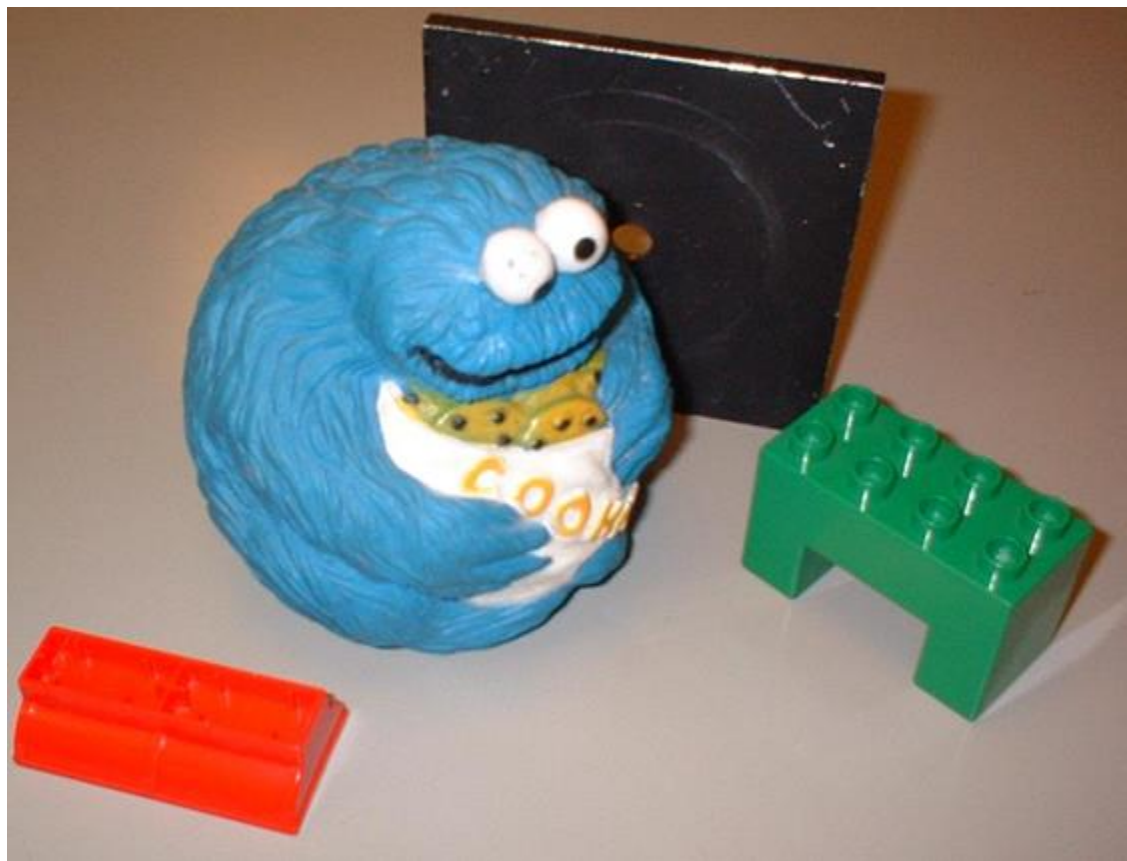
The histogram gives **the statistical distribution of image pixel intensity** values from which we can infer the dynamic range of the image and how well it is used.

Large range in the histogram means large dynamic range in the image.

Histogram Examples

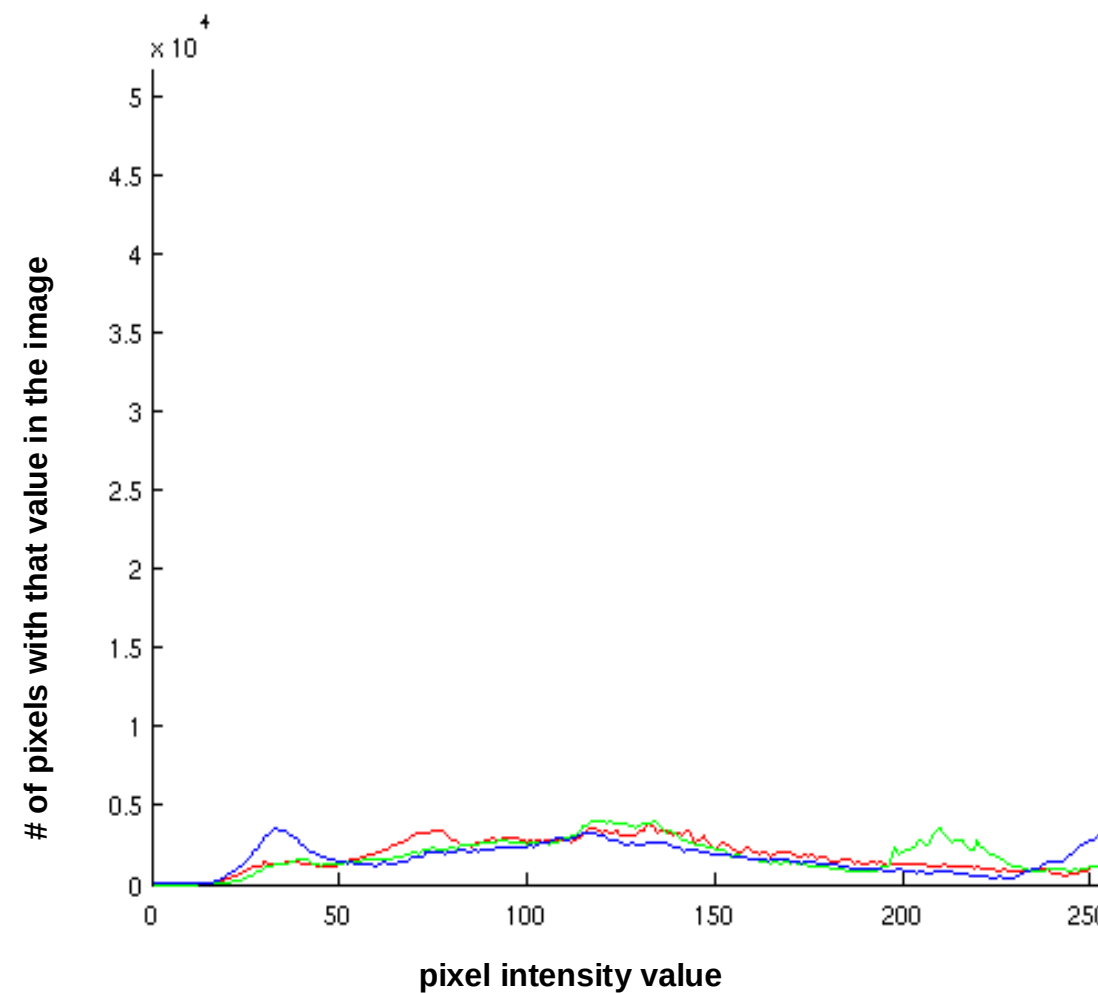


RGB Colour Histograms



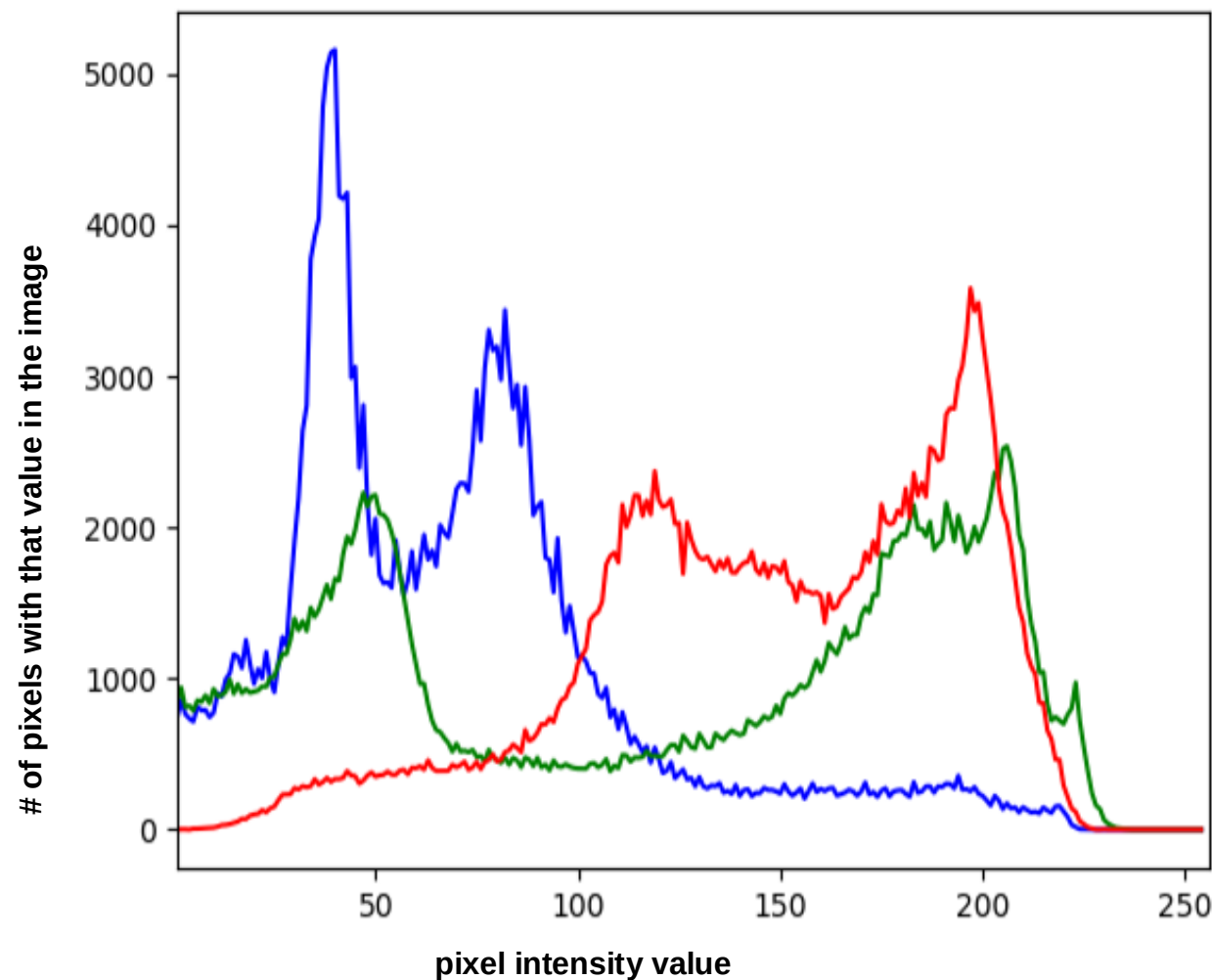
Extend to each R, G, B colour channel **separately**.

RGB Colour Histograms



More complex scenes give more complex pixel distributions in the histogram.

RGB Colour Histograms



Different scenes have different pixel distributions and histogram functions.

Histograms are functions describing global information extracted from the image.

We can use this information to perform **global image transforms**.

Histogram Transforms

We will look at three examples:

- **Contrast Stretch**
- **Histogram Equalization**
- **Histogram Specification**

Contrast Stretch

■ **Operation:** stretches the pixel range over a larger dynamic range.

– Also known as normalisation.

■ **Approach:** use four intensity values

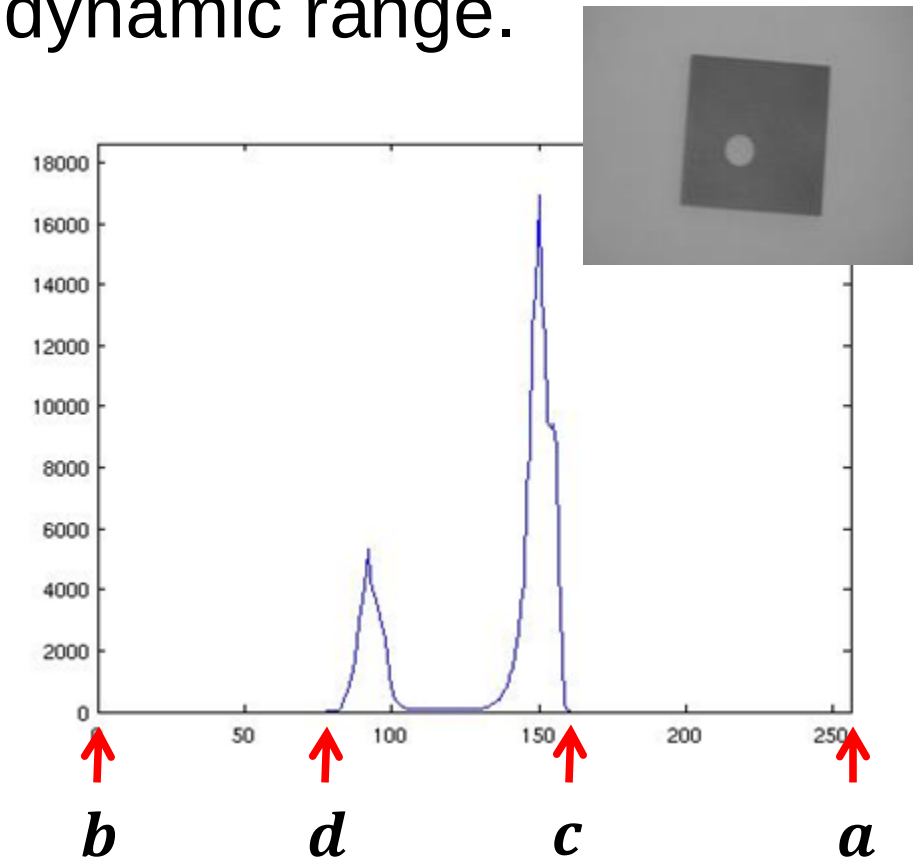
– a : upper pixel quantisation limit

– b : lower pixel quantisation limit

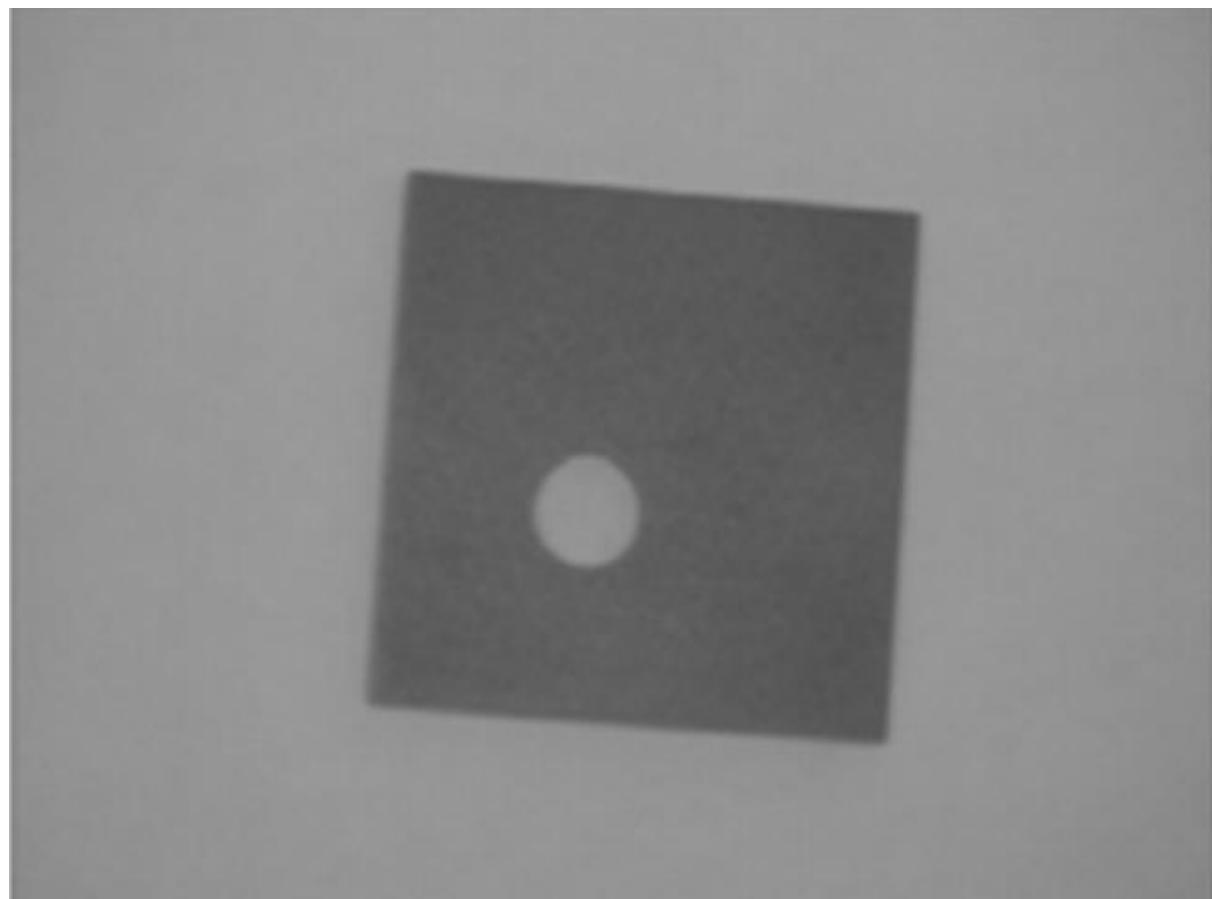
– c : maximum pixel value present

– d : minimum pixel value present

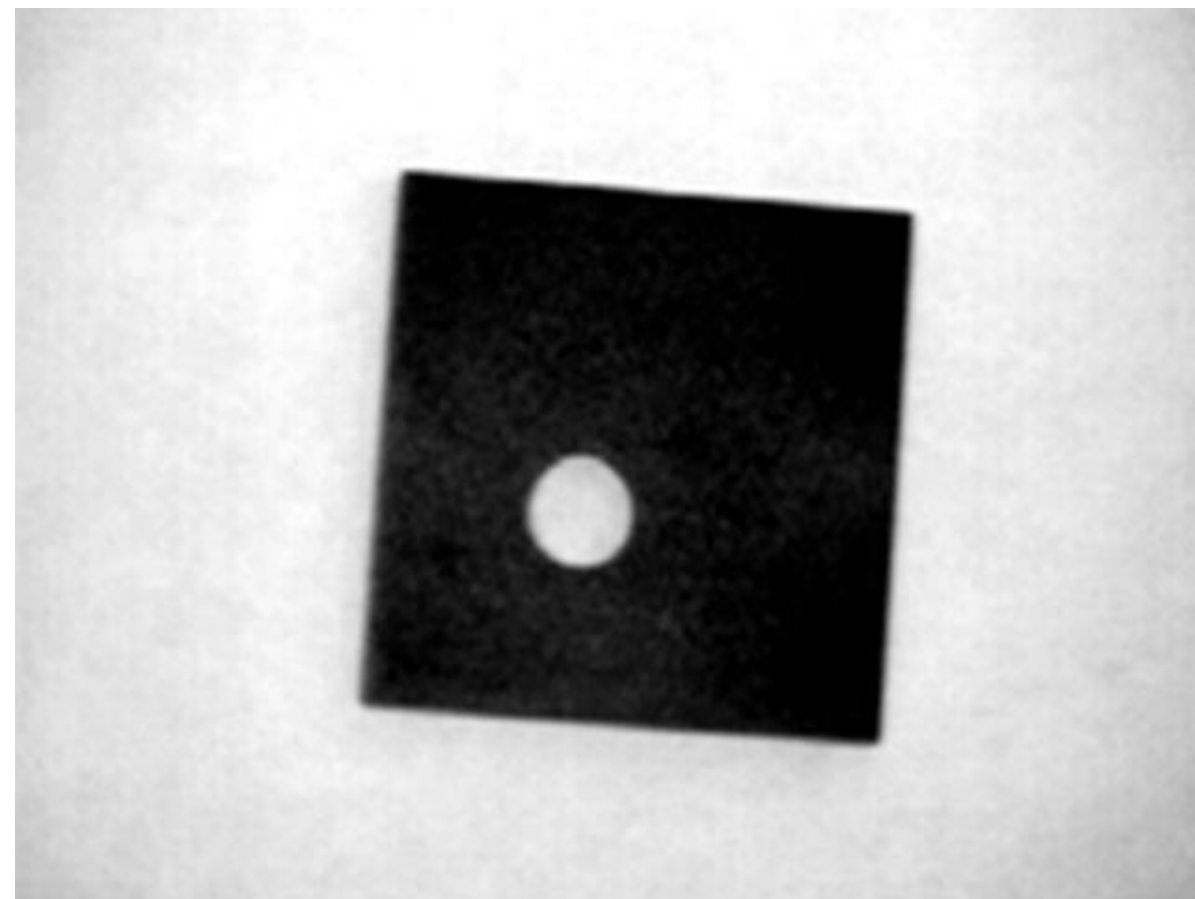
$$I_{\text{output}}(i, j) = (I_{\text{input}}(i, j) - c) \left(\frac{a - b}{c - d} \right) + a$$



Contrast Stretch Effect

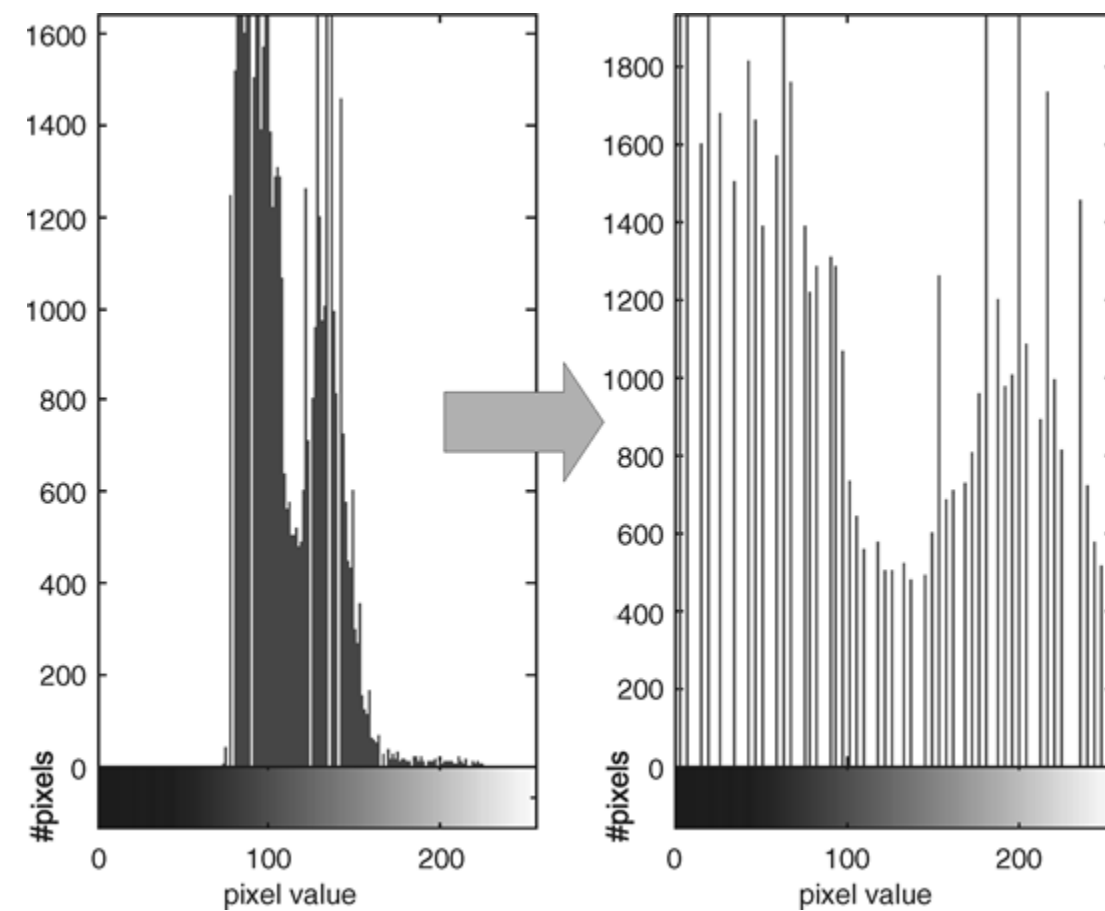


Input



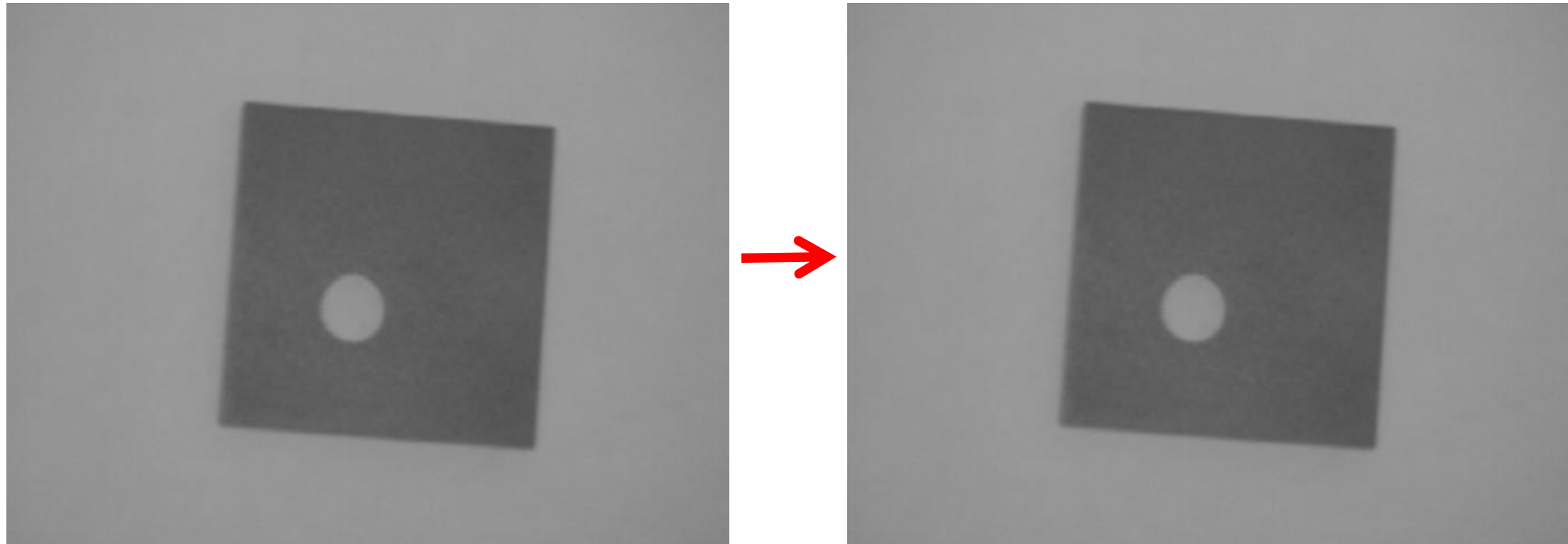
Output

Contrast Stretch Effect



Contrast Stretch - *Potential Problem*

- Outliers in image (e.g., salt and pepper noise)
 - possible that $c \sim = a$ and $d \sim = b$ (or even $==$)
 - **result:** contrast stretch has **no effect** on the image (check the math).



Contrast Stretch - *Solution*

Use a method robust against outliers to select c and d , instead of the min and max of the pixels values in the image.

Selection Method 1:

- Compute the histogram of the input
- Select c and d at **fixed percentiles points** of the histogram distribution.

For example, select c and d at the 5th and 95th percentile points of distribution, which means that 5% of pixels will be less than c and 5% greater than d .

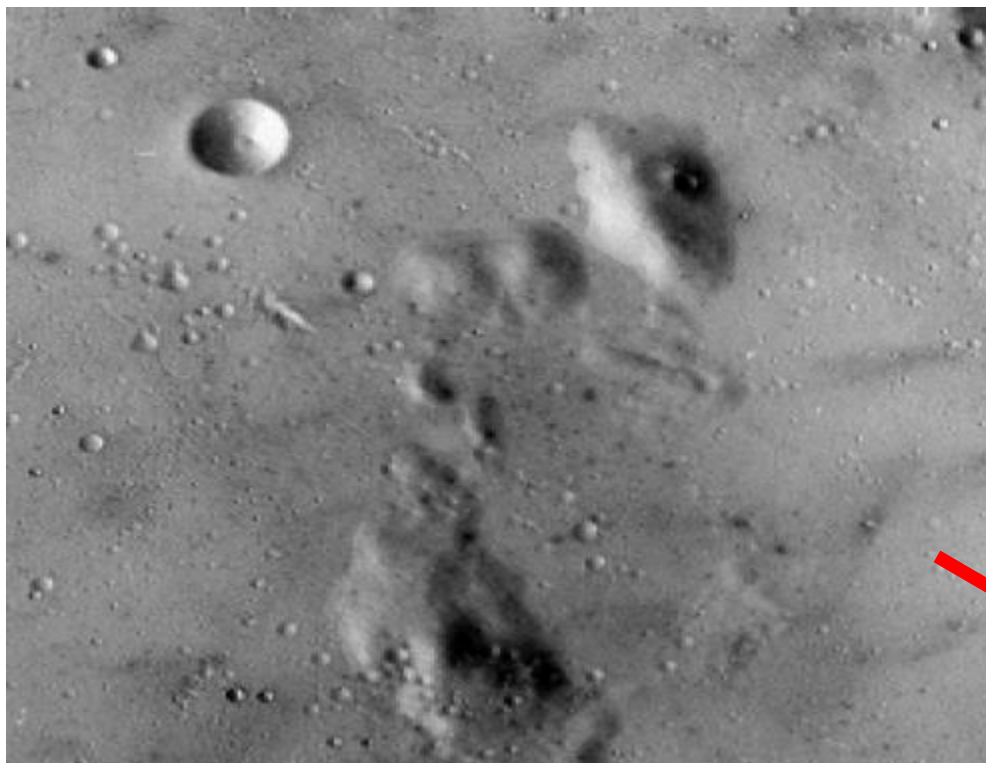
Contrast Stretch - *Solution*

Selection Method 2:

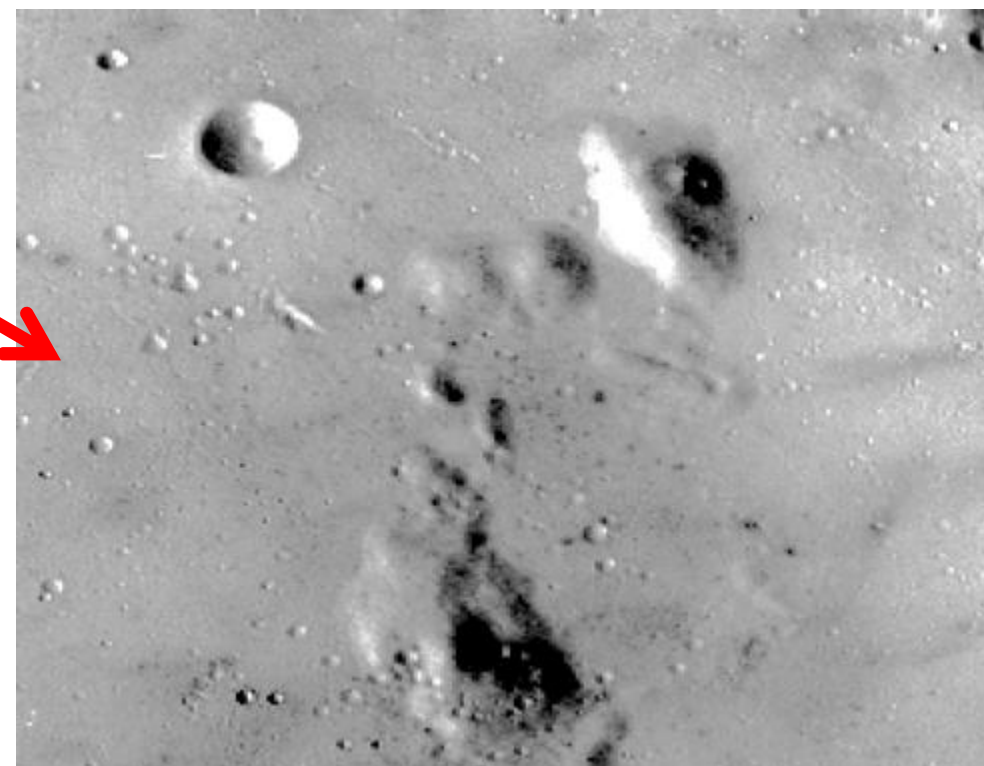
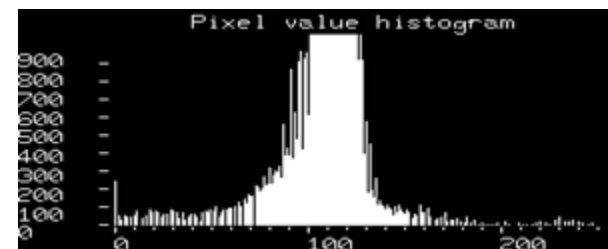
- find the most frequent image value (histogram peak)
- select a cut-off as a percentage of the peak
- scan down from peak in either direction until last values above cut-off are reached and select these as c and d .
- if any of new intensity values are below b or above a , map them to b and a respectively.

N.B. Method 2 is a bit more elaborate but also marginally weaker for complex, multi-peak histograms.

Examples

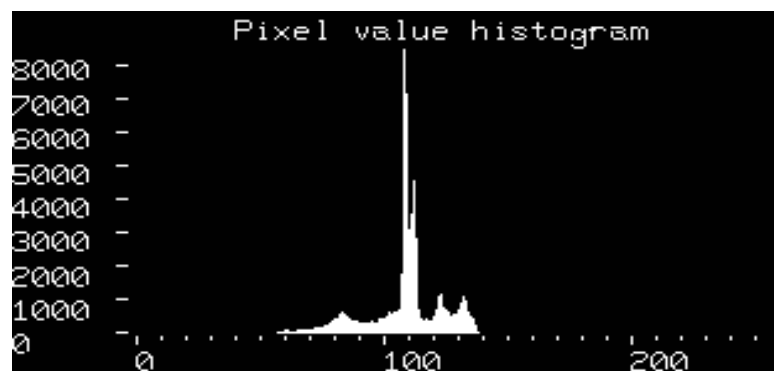


source: HIPR2 © 2003/4

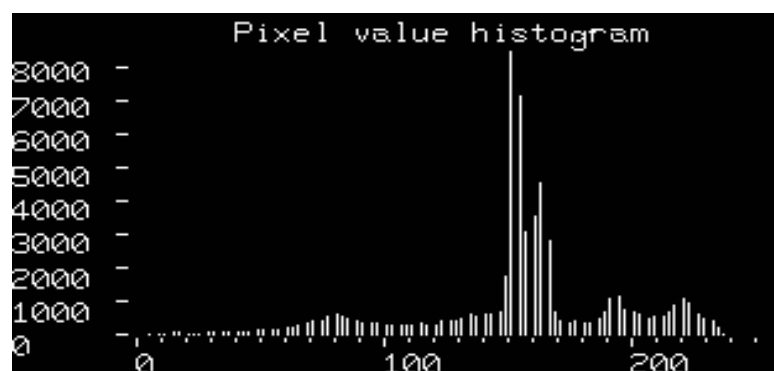


Here c and d are at the 1st and 99th percentile.

Examples

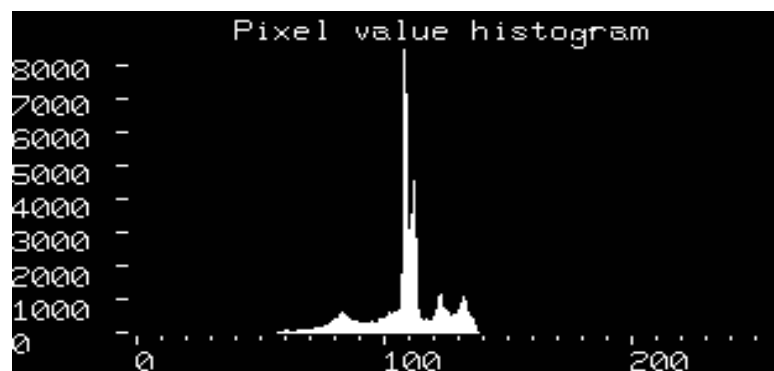


Using $c = 79$ and $d = 136$



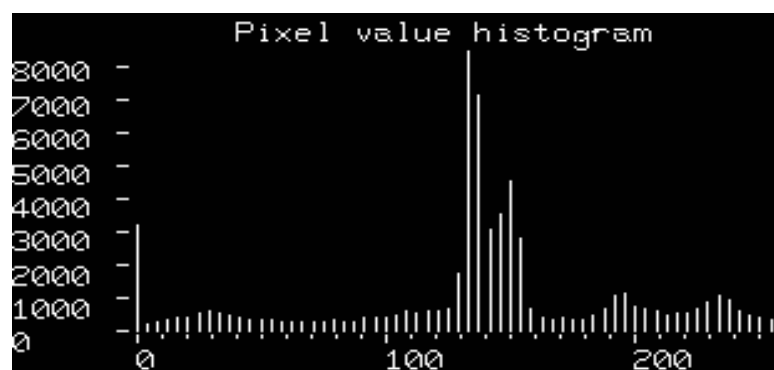
Result still a bit flat

Examples



Cut-off fraction 0.03

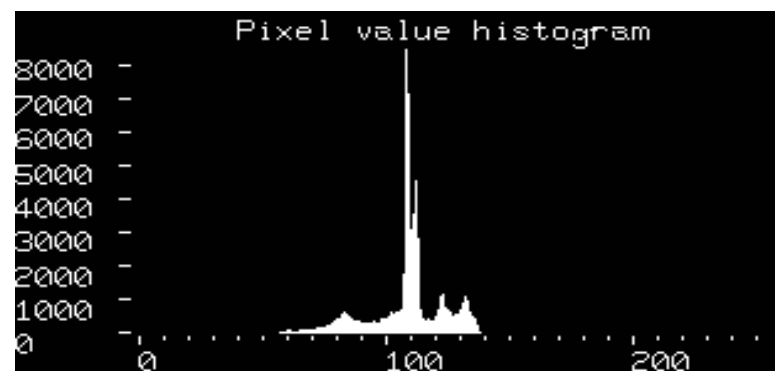
Information did spread out



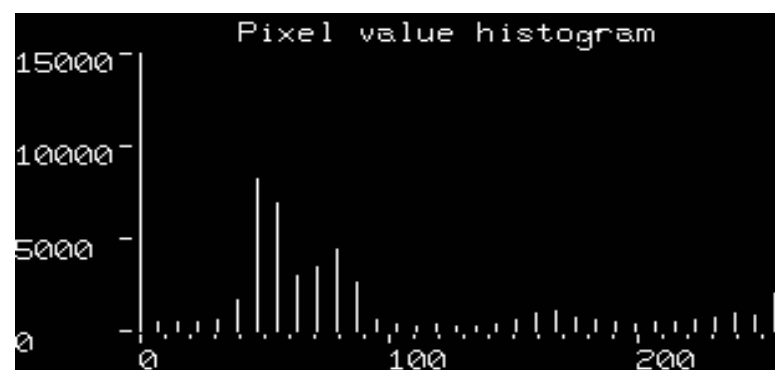
Values to left of the peak were pushed towards zero.

Source: HIPR2

Examples



cut-off fraction 0.125



Values to left of the peak were largely mapped to zero resulting in a darker image.

Source: HIPR2

Quick Demonstration

- Simple Contrast Stretching:

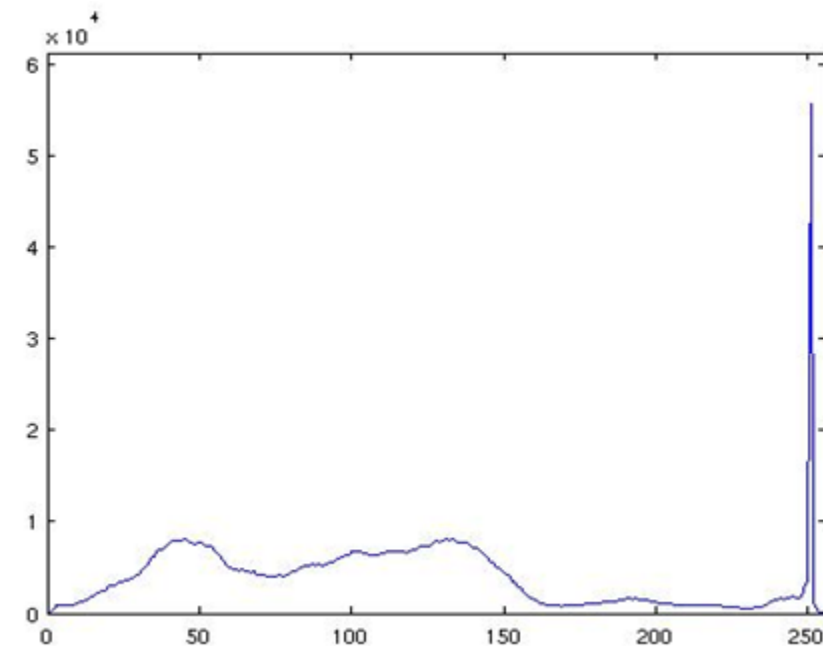
https://github.com/atapour/ip-python-opencv/blob/main/contrast_stretching.py



Histogram Equalisation

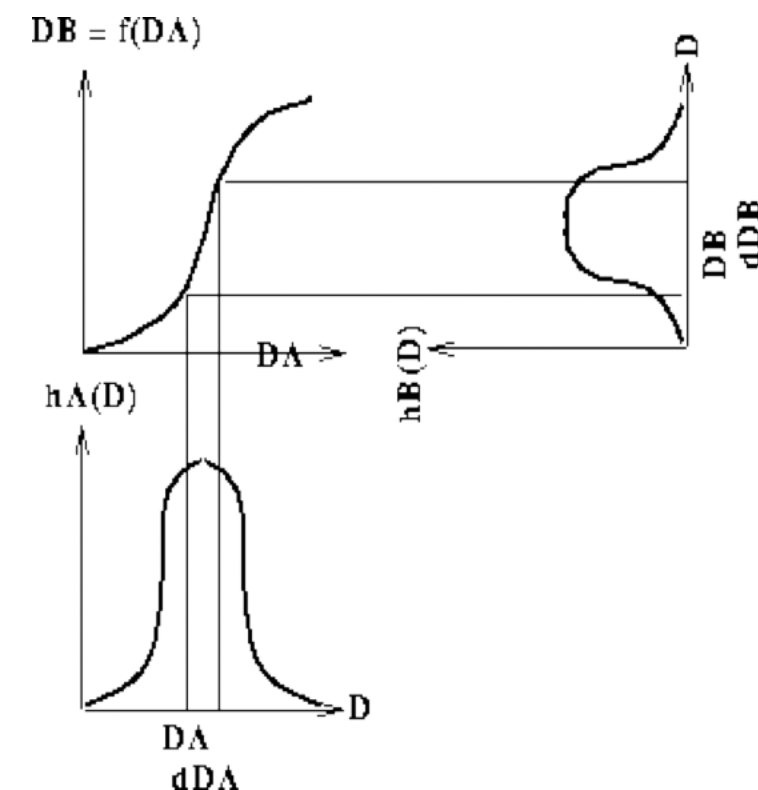
Histogram Modelling: modify an image so that its histogram conforms to a given shape.

Histogram Equalisation: histogram modelling via a **monotonic, non-linear** intensity transformation function aiming at producing an output image with **uniform** histogram distribution.



Histogram Equalisation

- **Map pixels** from image A to B using a single-valued and monotonically increasing function $f()$.
- e.g. transform the density DA to DB and similarly densities in the range $DA \rightarrow dDA$ to $DB \rightarrow dDB$



Cumulative Histogram Function

- each histogram entry = density of a given intensity value
 - a normalised histogram = probability distribution of image intensity
 - divided by the total number of the pixels of the image
- From histogram $h(i)$ – i being the dynamic range of the image, we construct the **cumulative histogram function** $C(i)$ - a monotonically increasing function:

$$C(i) = \sum_{j=1}^i h(j)$$

That is, the values of $C(i)$ record the sum of the occurrence of each grey level up to and including i .

Histogram Equalisation

- $C(i)$ records the frequency of occurrence all grey levels up to and including $h(i)$.
- $C()$ = **single valued, monotonically increasing function.**
 - *In (very) simple language - **an increasingly additive curve!***
 - Gradient of $C()$ $\sim$ = current equalisation of the image
 - *A constant gradient of 1 will be a perfectly equalised image (the increase at each step is constant)*

Histogram Equalisation

In an **ideally equalised** image, all intensity values would appear the same number of times, i.e. N/L each, where N is the number of pixels in the image, L the maximum possible intensity level (*255 for an 8-bit grayscale image*).

- The cumulative function would then be

$$C_{\text{ideallyEqualised}}(i) = (N / L) \cdot i$$

Histogram equalisation corresponds to the intensity transform

$$t(i) = (L / N) \cdot C(i)$$

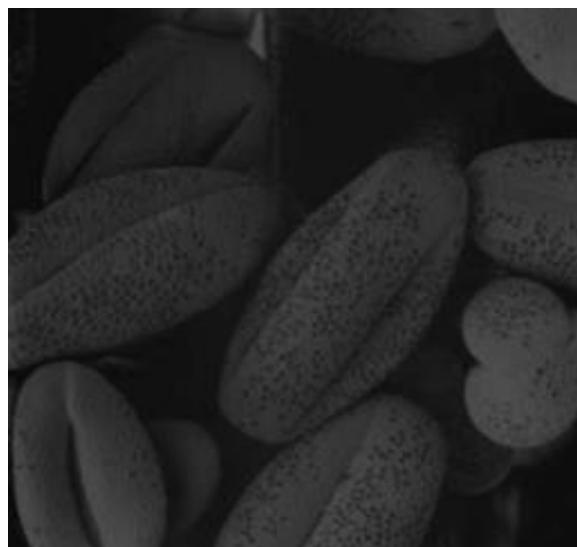
which computes the cumulative histogram at intensity i , and maps i to the intensity of the ideally equalised image for that value of the cumulative histogram.

Histogram Equalisation

An **schematic example** of how histogram equalisation works:

- Let $L = 100$ (dynamic range of 100 levels)
- Let $C(50) = 0.8 \times N$ (80% of the N pixels have value of 50 or lower)
- $t(50) = \left(\frac{L}{N}\right) \times 0.8 \times N = 100 \times 0.8 = 80$
- 50 is mapped to 80 and thus, 80% of the pixels of the equalised image have a value of 80 or lower.

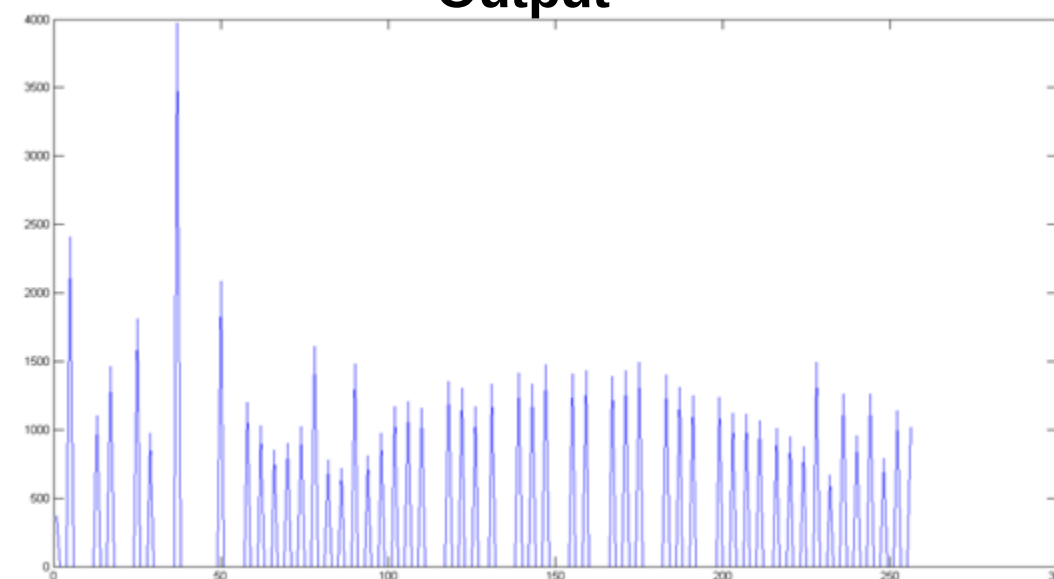
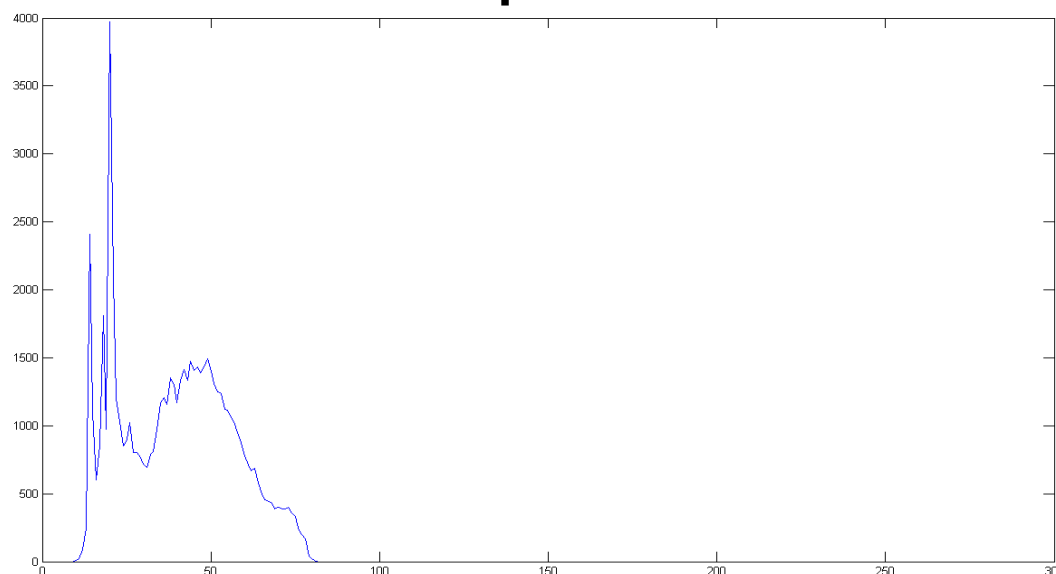
Histogram Equalisation - Examples



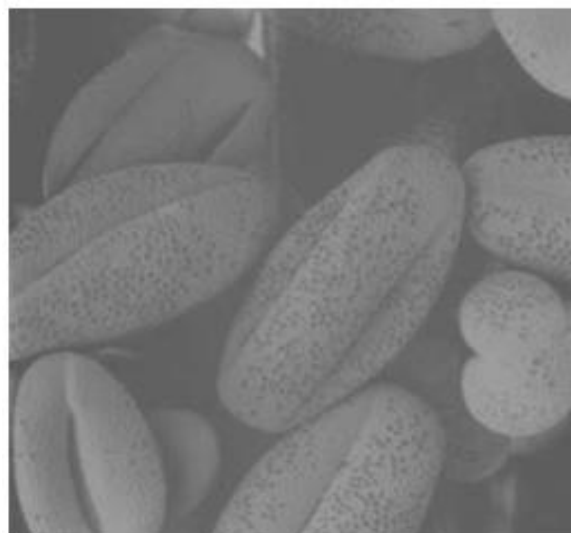
Input



Output

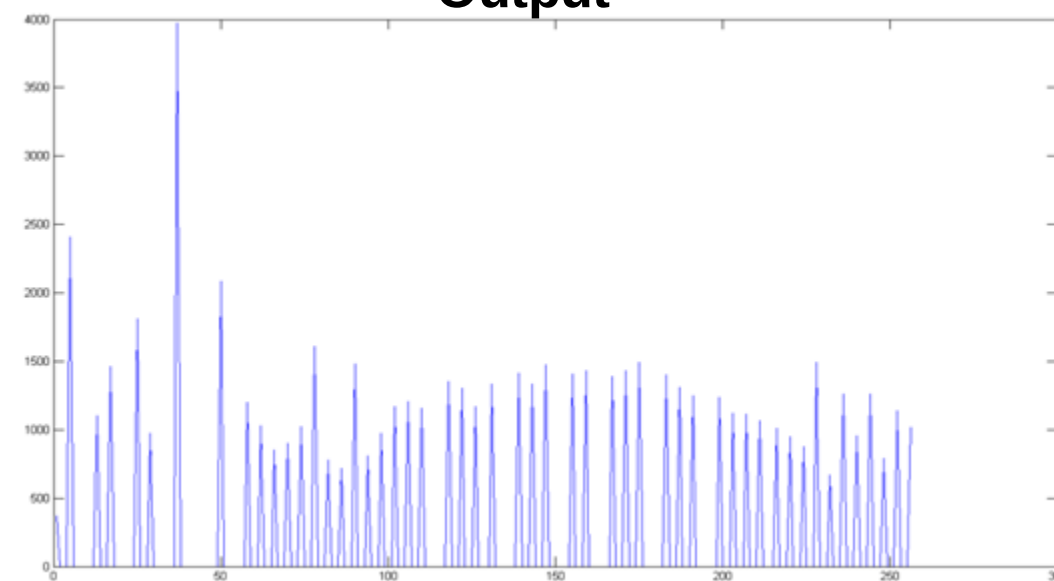
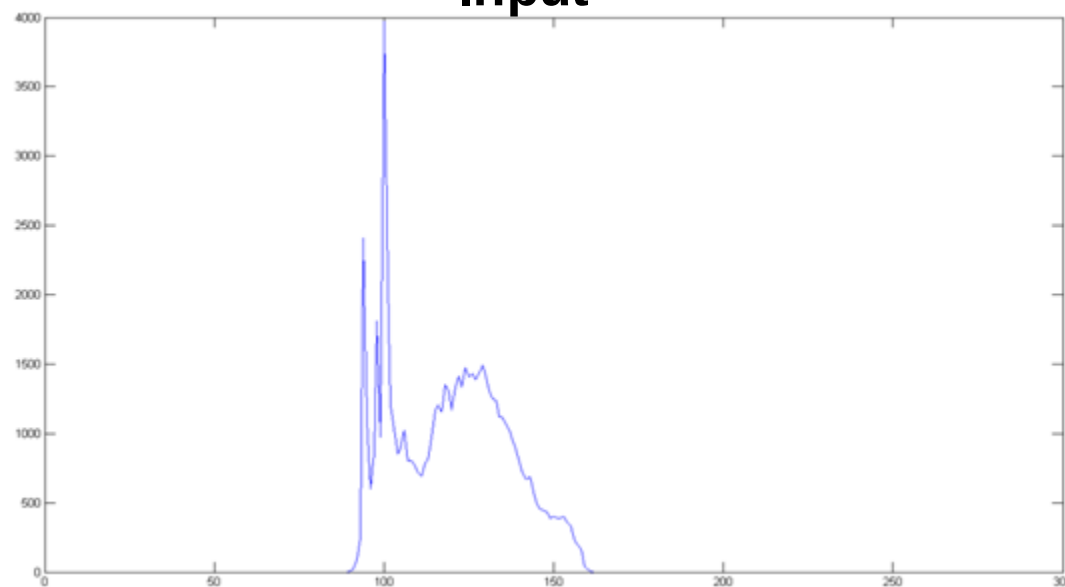


Histogram Equalisation - Examples



Input

Output



Histogram Equalisation

- Histogram equalisation is a **point intensity transform** and can be efficiently encoded into a look-up table (LUT) mapping values in I_{input} to I_{output}
- The output image is not necessarily fully equalized and there may be “**holes**” in the histogram (i.e. unused intensity levels).
 - The discretisation of the problem (e.g. 256 intensity levels) introduces discontinuities.

Full equalisation is possible by swapping pixels between intensity levels, i.e., re-arranging pixels (not covered here).

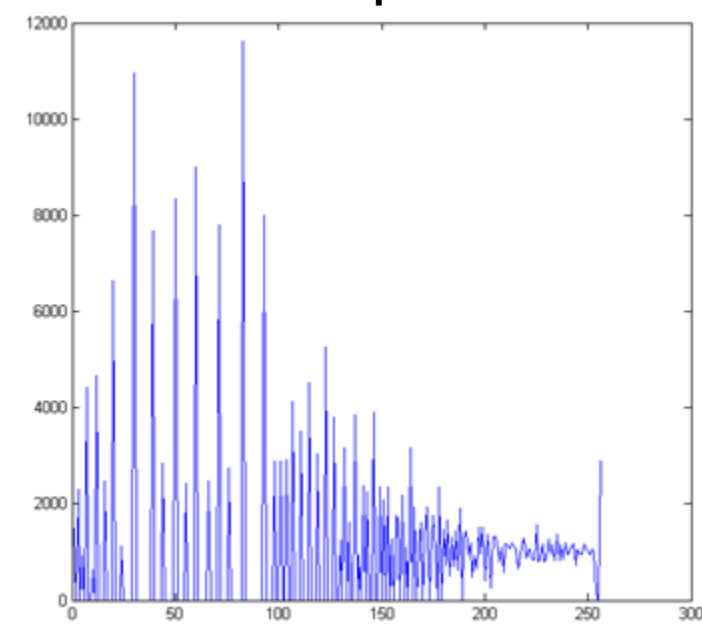
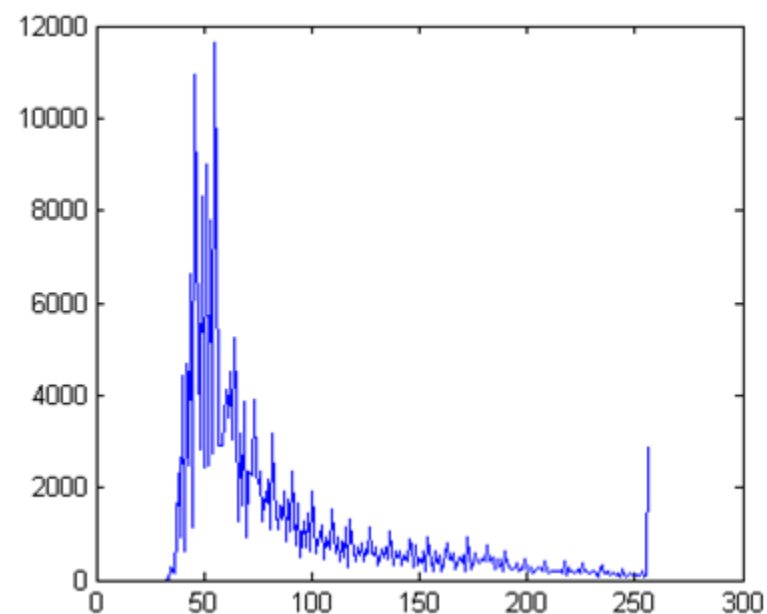
More Examples



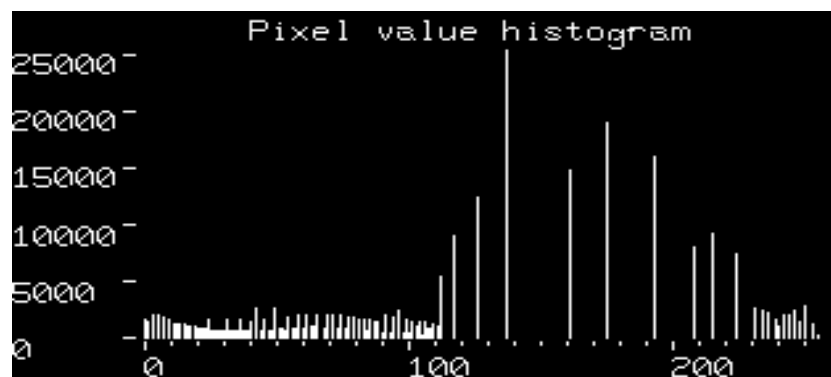
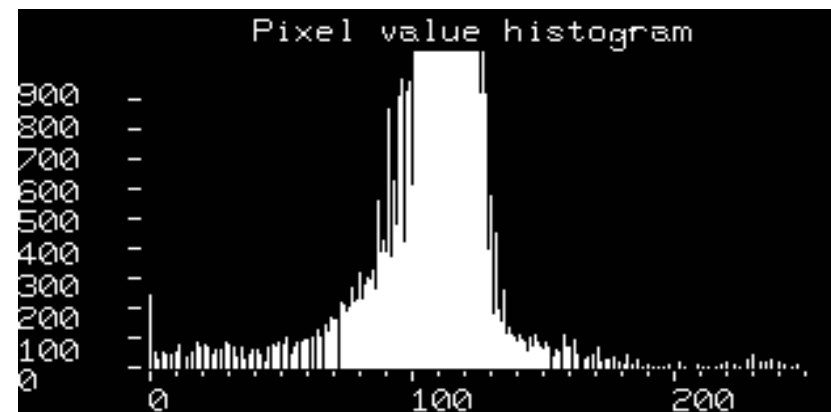
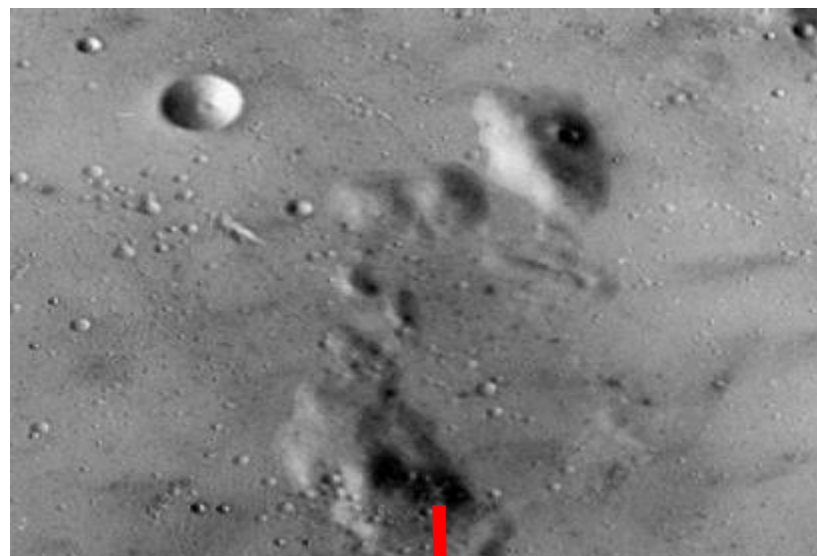
Input



Output

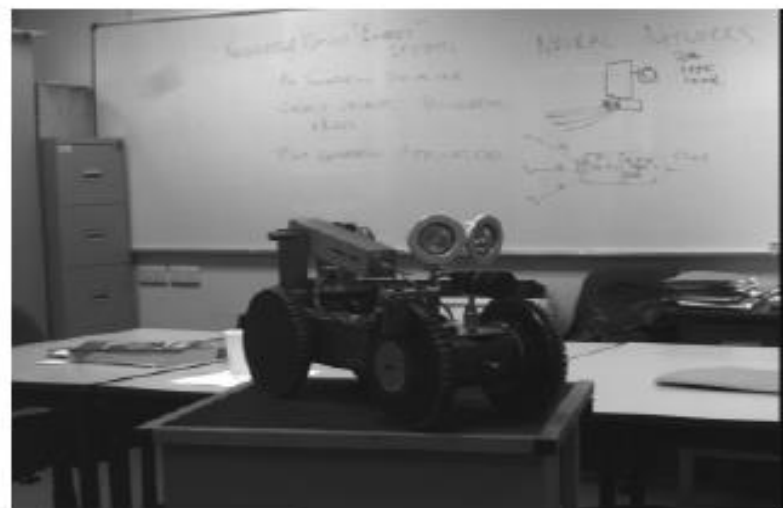
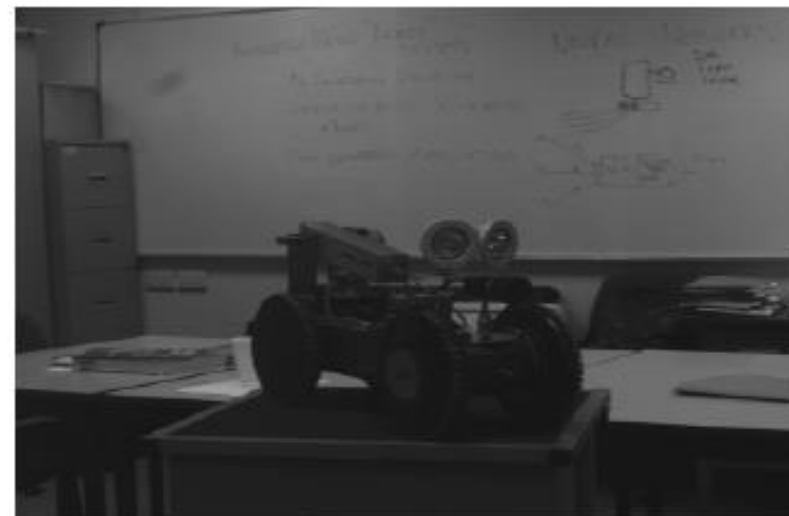


More Examples



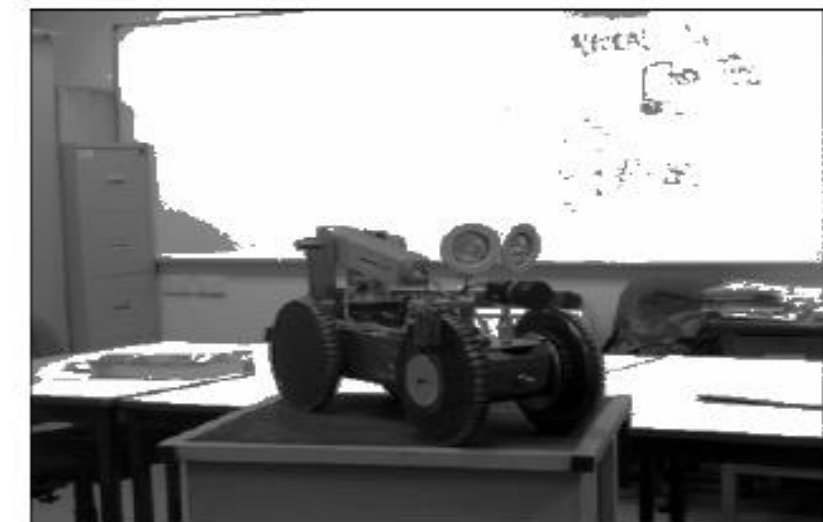
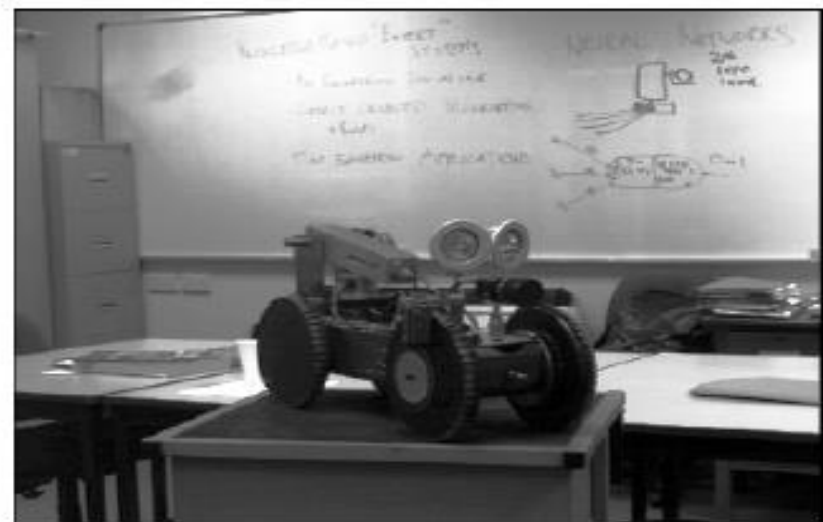
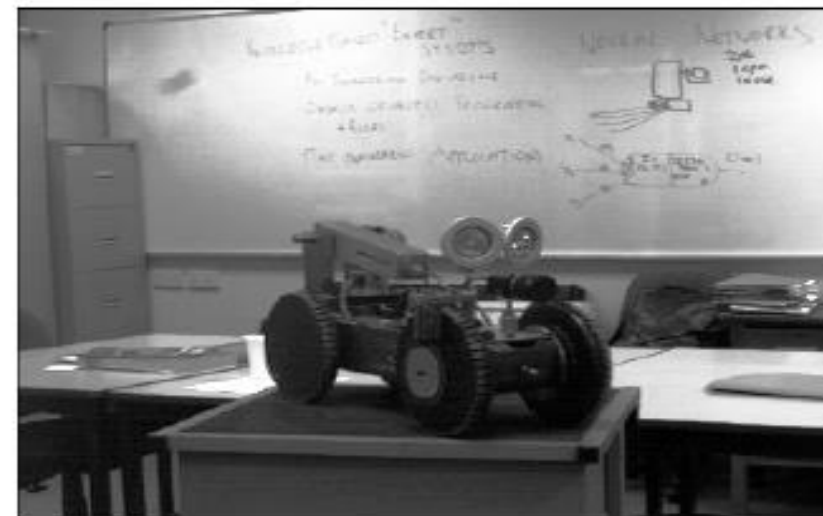
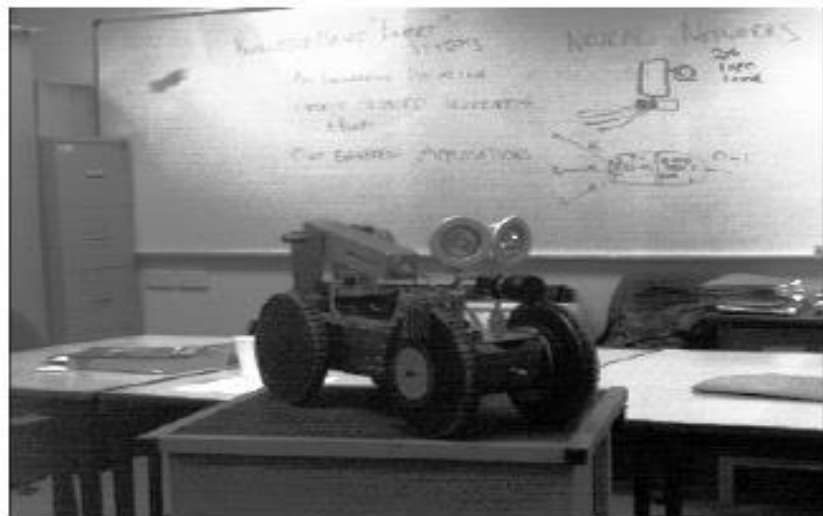
source: HIPR2 © 2003/4

More Examples - Inputs



Original images (different exposures)

More Examples - Outputs



Histogram equalization results

Application: *Assignment* - maybe



Original Input

Histogram Equalized Output

You don't have to use this, but it is a good starting point!

Quick Demonstration

- Histogram Equalisation:

https://github.com/atapour/ip-python-opencv/blob/main/equalise_histogram.py

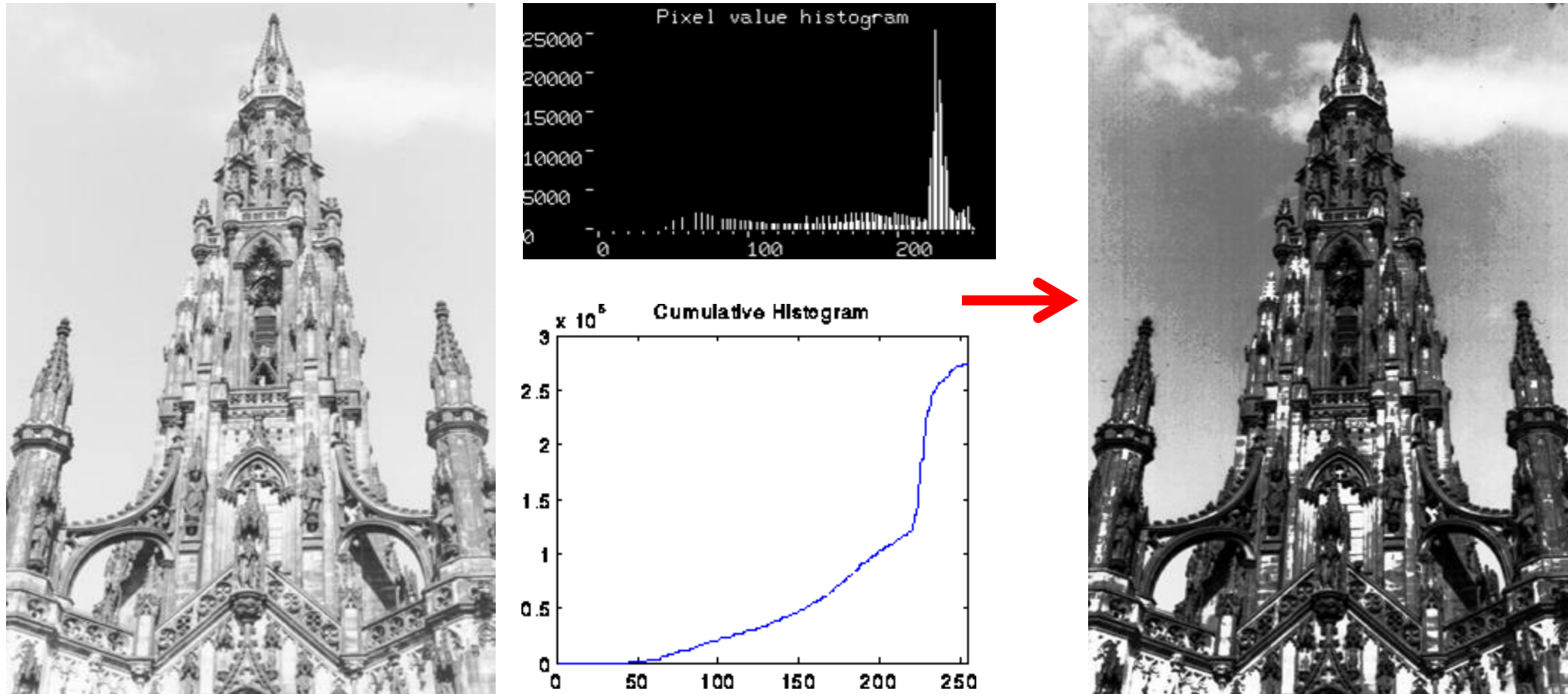


Limitations

As a fully automated technique (with no parameters), the effect of histogram equalisation is highly input dependant.

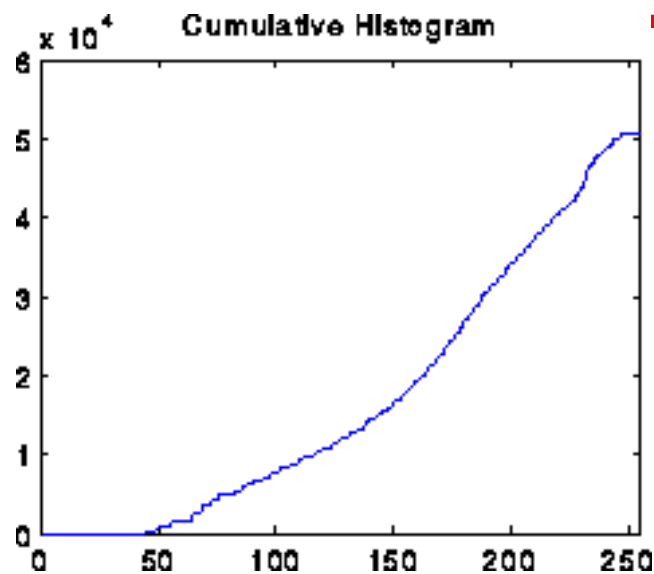
In some images the global contrast can be over-exposed or under-exposed.

Histogram Equalisation – *Poor Example*



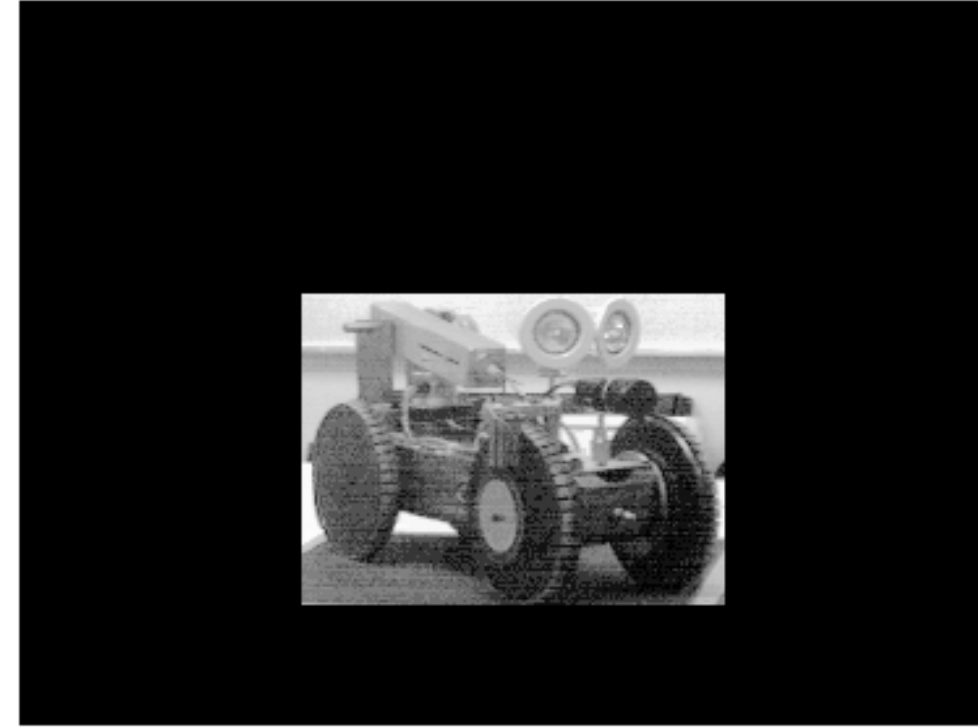
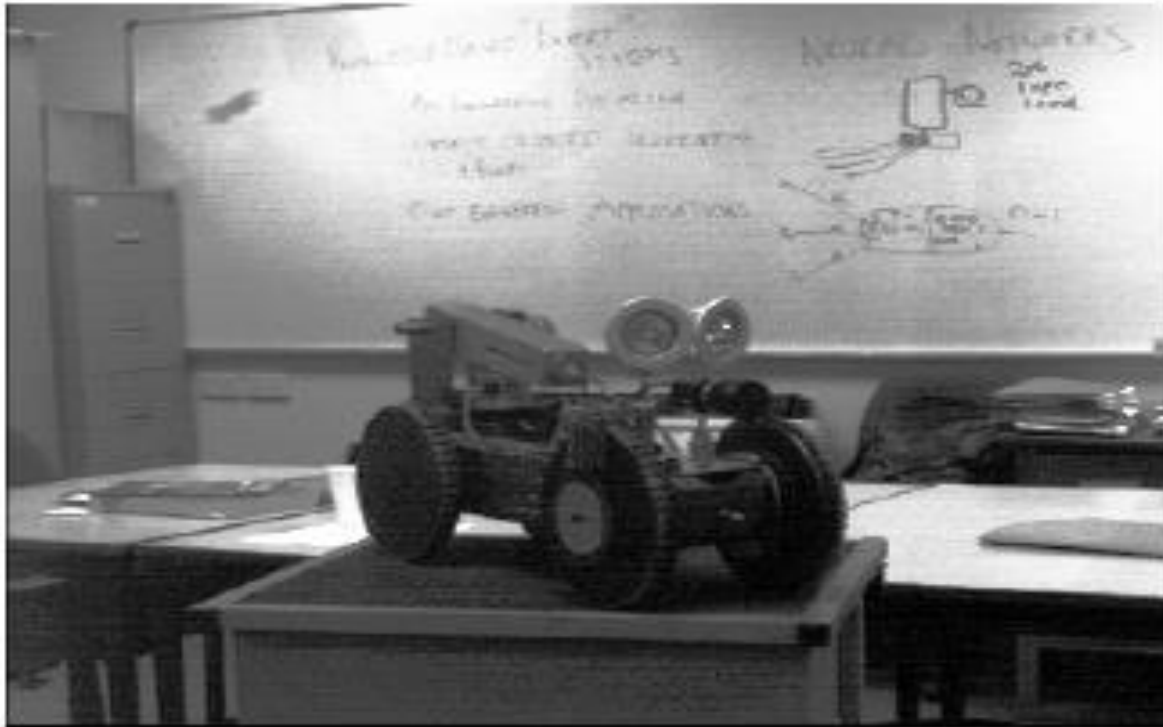
The histogram equalised result has **poor dynamic range in the $\sim 100 - 230$** range because the cumulative histogram has shallow gradient in this range.

Histogram Equalisation – *Potential solution*



Solution: use the histogram from a well-balanced sub-part of original image as the input histogram of the equalisation algorithm.

Localised Histogram Equalisation



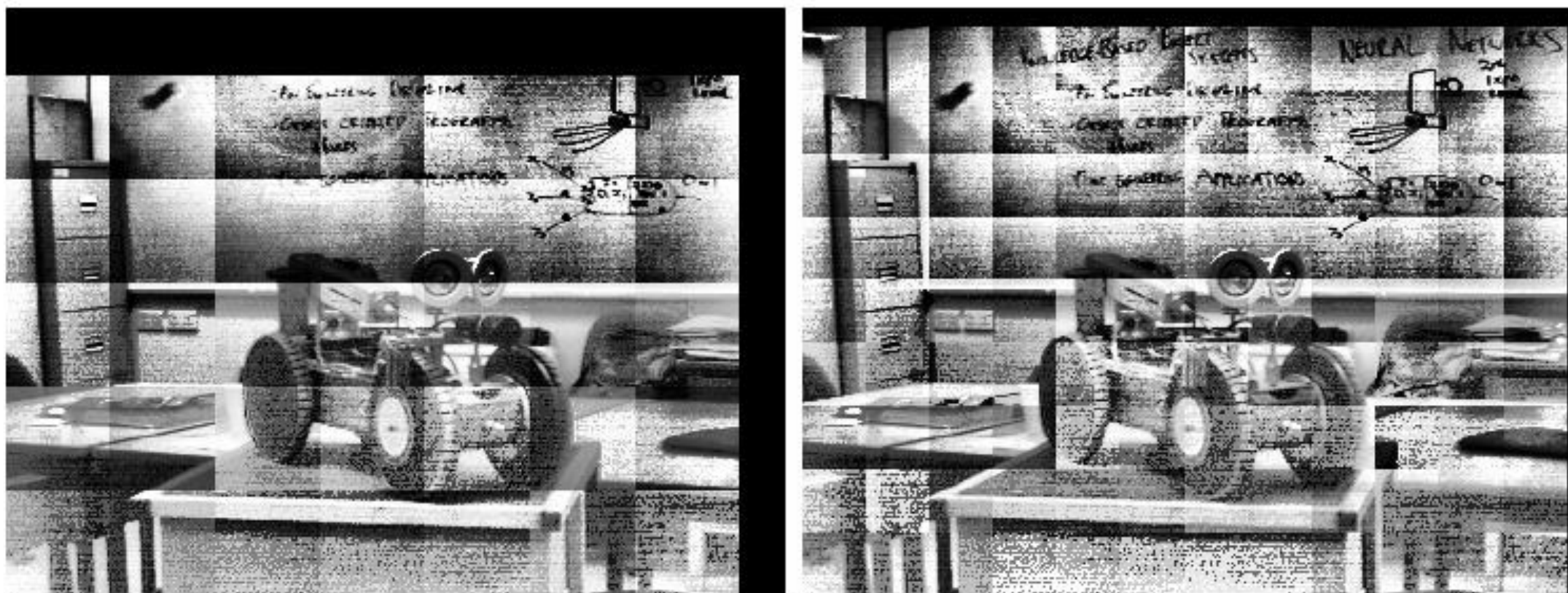
Standard histogram equalisation performed within just a sub-part of the image.

The computation of the histogram uses values from the **area of interest** only.

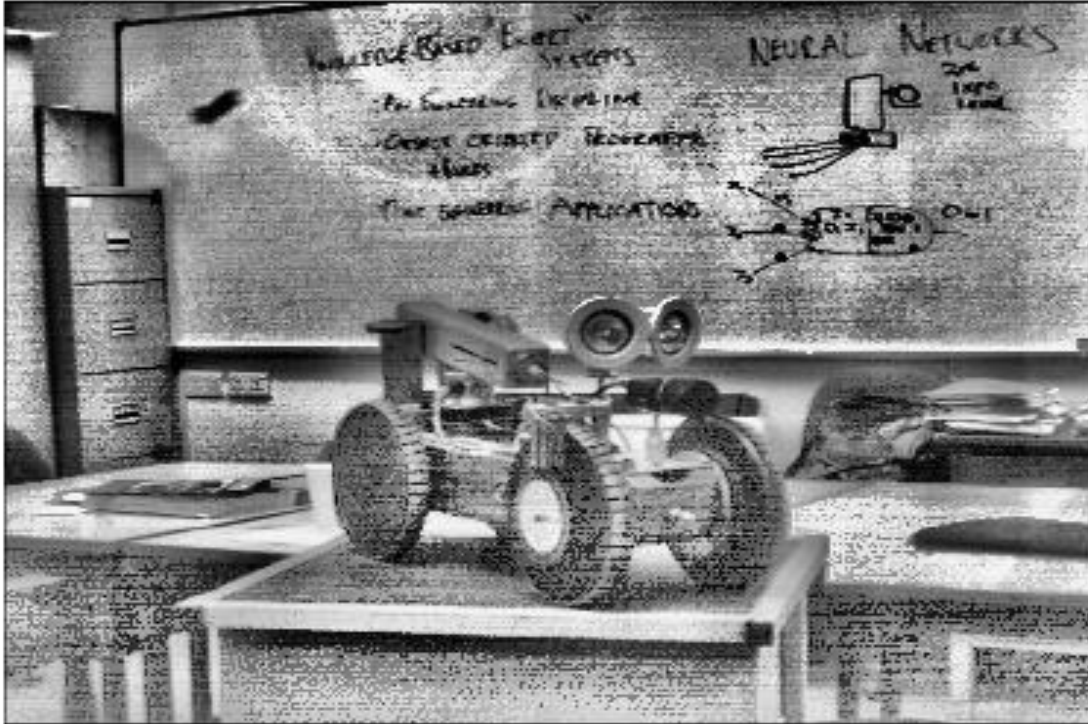
Localised Histogram Equalisation

Split the image into a set of discrete, non-overlapping windows of size $N \times N$.

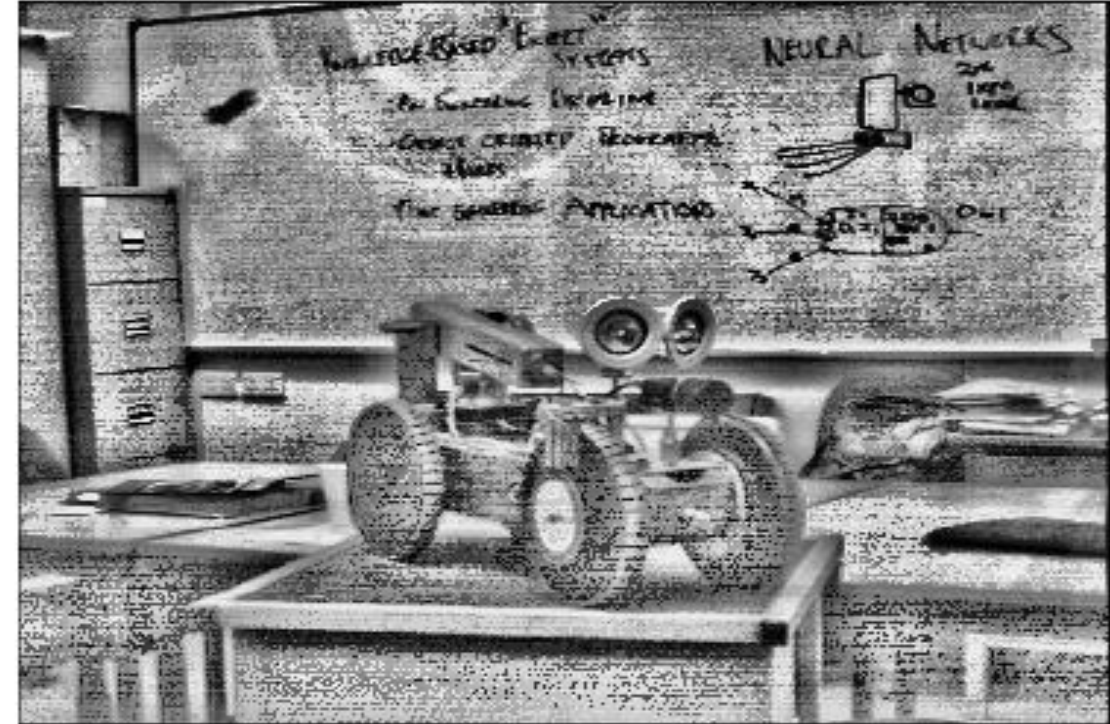
- Histogram equalisation of each neighbourhood in isolation (**tiling**).



Adaptive Localised Histogram Equalisation



window size = 51 by 51

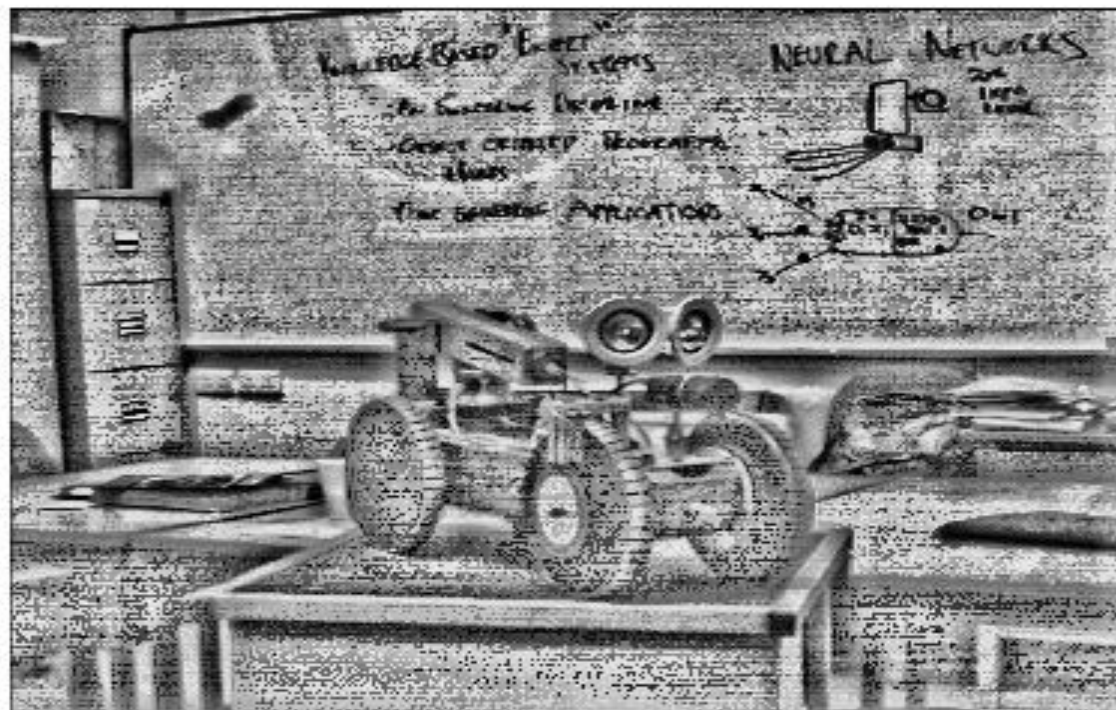


window size = 31 by 31

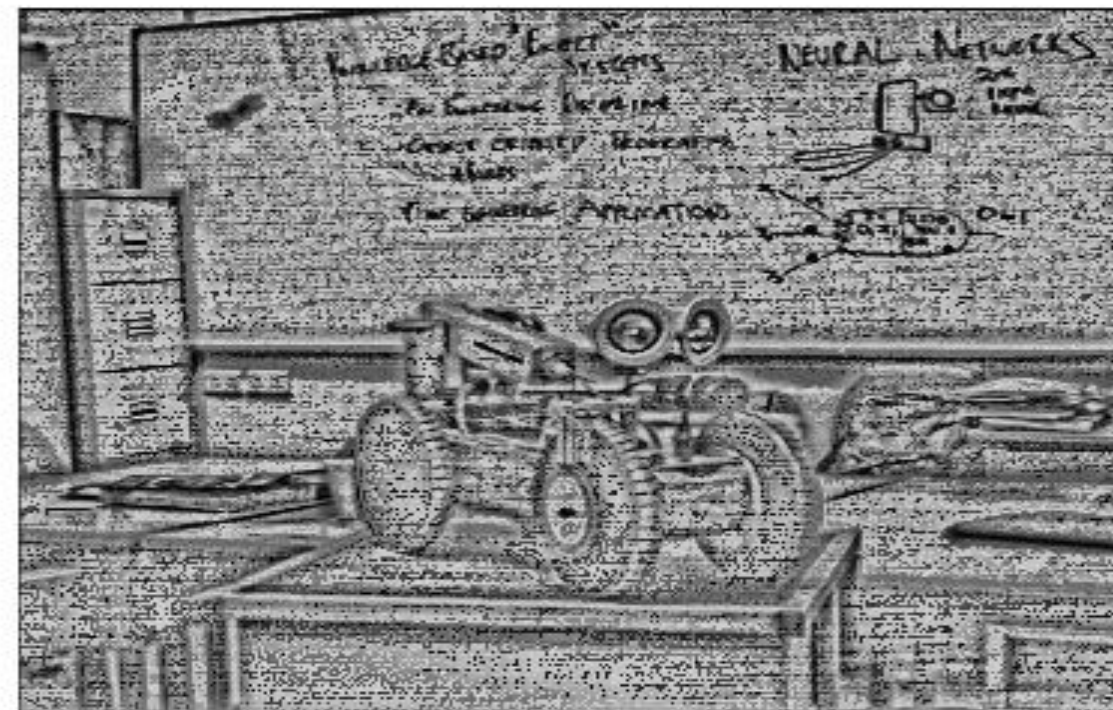
Histogram equalisation using **overlapping** local $N \times N$ neighbourhoods.

- Adaptive histogram equalisation is slower than localised histogram equalisation.

Adaptive Localised Histogram Equalisation



window size = 21 by 21



window size = 11 by 11

Tiling artifacts (of the localised histogram equalisation) can be avoided but the choice of **neighbourhood size N** is crucial.

Localised + Adaptive Equalisation

In terms of the aesthetics of the output, performance is **poor** because a global transform (of the entire dynamic range) is computed and applied locally.

Solution ...

Localised + Adaptive Equalisation

In terms of the aesthetics of the output, performance is **poor** because a global transform (of the entire dynamic range) is computed and applied locally.

Solution ...

CLAHE - Contrast Limited
Adaptive Histogram Equalisation

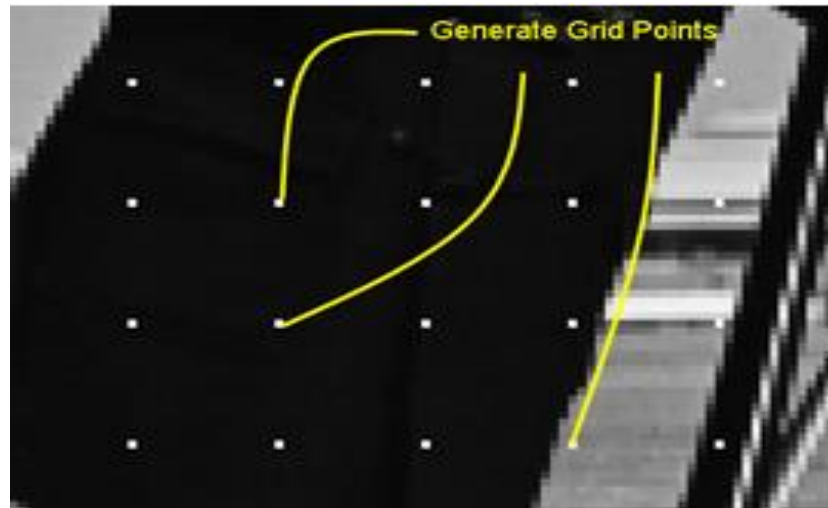
CLAHE

A method of **controlled** equalisation:

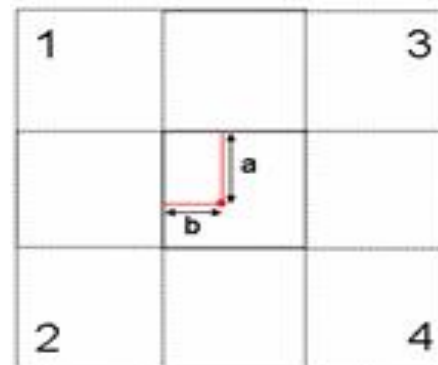
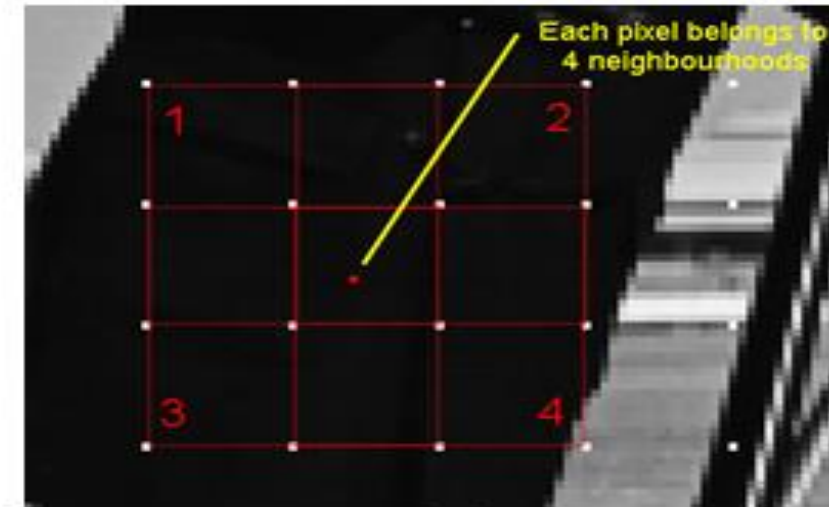
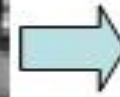
- Create a regular grid over the image with spacing W .
 - At every grid node, consider a $2W \times 2W$ window, consisting of 4 square elements of the grid.
 - Note the 50% window overlap.
 - Calculate the histogram equalisation “**look-up table**” for **each window**.
 - Due to the overlap, **each pixel lies in 4 windows**, thus has 4 corresponding lookup values.
- The output is **a linear combination of these 4 values**.



CLAHE

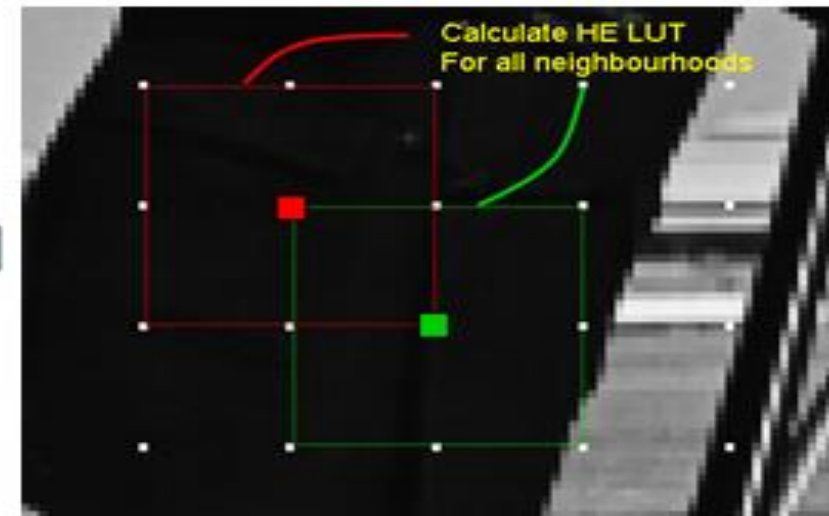
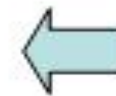


Solomon/Breckon '10



Each of the 4 overlapping regions is independently histogram equalised. The pixel value at the point $\bullet$ is a *weighted* combination of the 4 values from the HE look-up-tables derived for each of the regions 1-4.

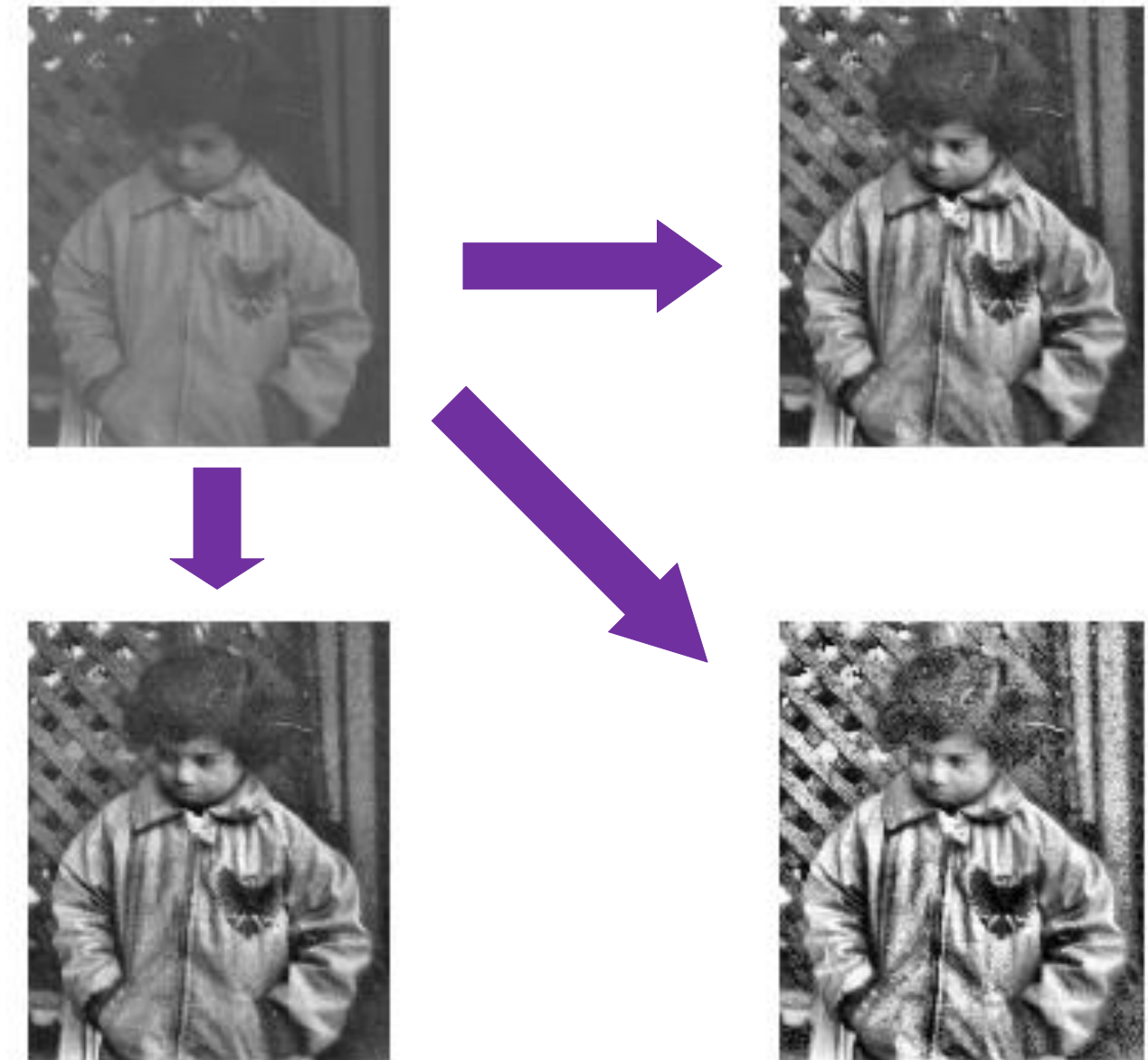
$$I = (1-a)(1-b)I_1 + a(1-b)I_2 + (1-a)bI_3 + abI_4$$



$0 < a, b < 1$ - I_1, I_2, I_3 and I_4 are the histogram-equalised look-up-table values.

CLAHE

- Finally we place a limit L on the overall (normalised) contrast gain in any given region
 - $0 < L < 1$
 - contrast gain = spread of pixel values within the region
 - i.e. dynamic range
- A method of **controlled equalisation**.
 - via **tunable parameters**.



Quick Demonstration

- CLAHE:

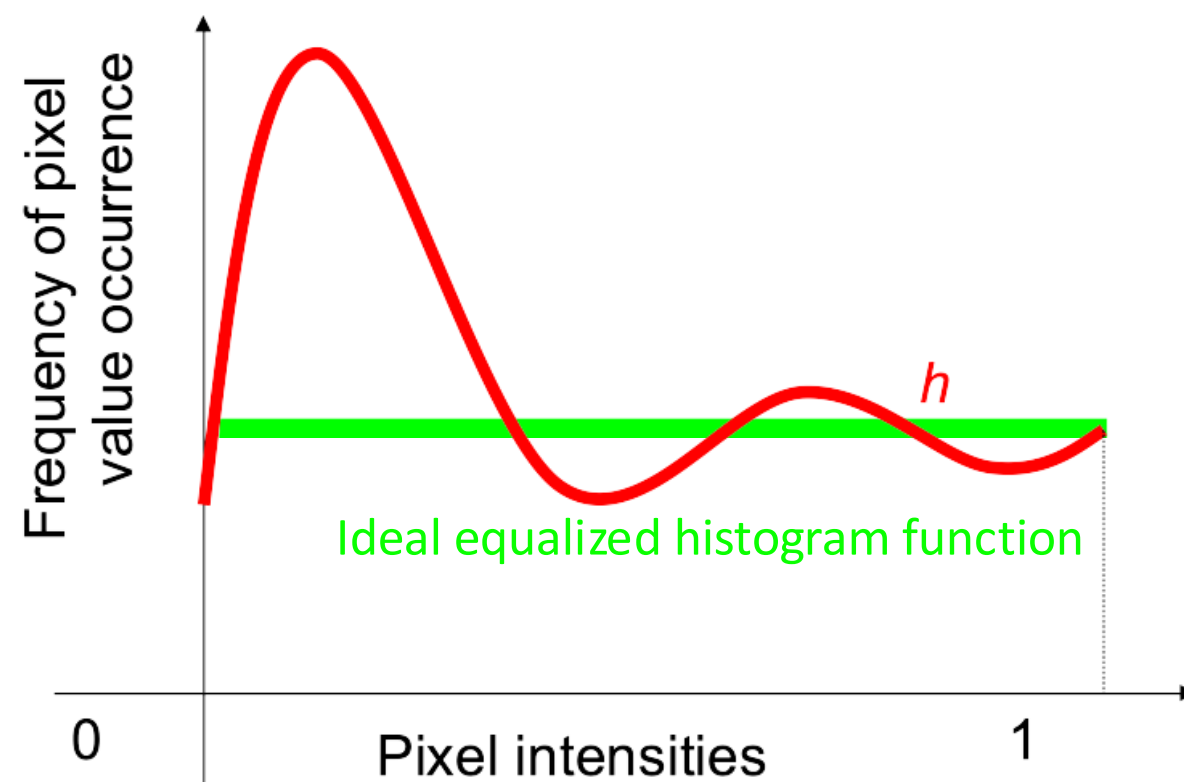
https://github.com/atapour/ip-python-opencv/blob/main/equalise_clahe.py



Histogram Specification

Generalisation of Equalisation: specified histogram distribution.
– target histogram distribution for our image, $h_t()$.

- In general, histogram function h can be seen as a continuous function.
- Histogram equalization tries to make the intensity distribution uniform.

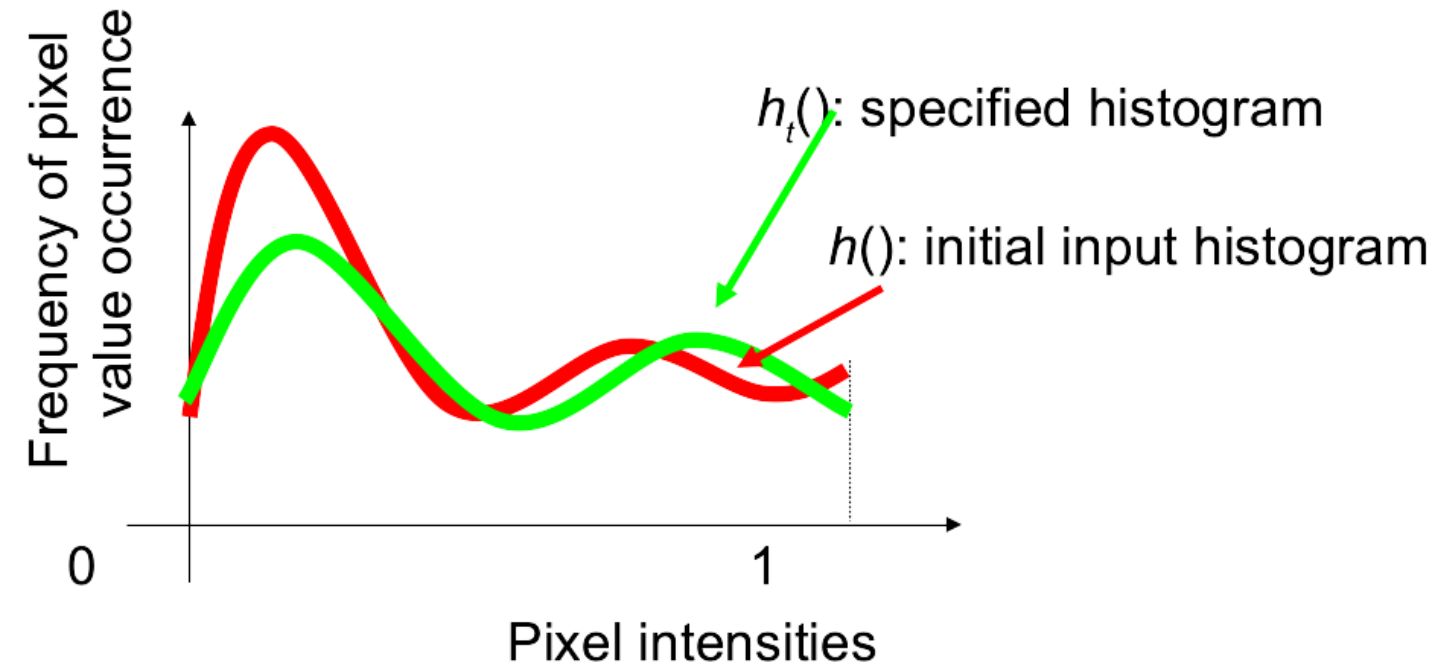


Histogram Specification

■ Now, we specify the function we want for the histogram.

Application in Image Enhancement:

This is how many complex image enhancement products/plugins work within mobile apps + Adobe Photoshop, etc. [at least before the boom of machine learning].



— The “secret sauce” = the distribution - $h_t()$? (different for each effect)

Summary

Histograms

- definitions and construction
- basic properties

Histogram Transforms

- contrast stretch
- histogram equalisation
- histogram specification

